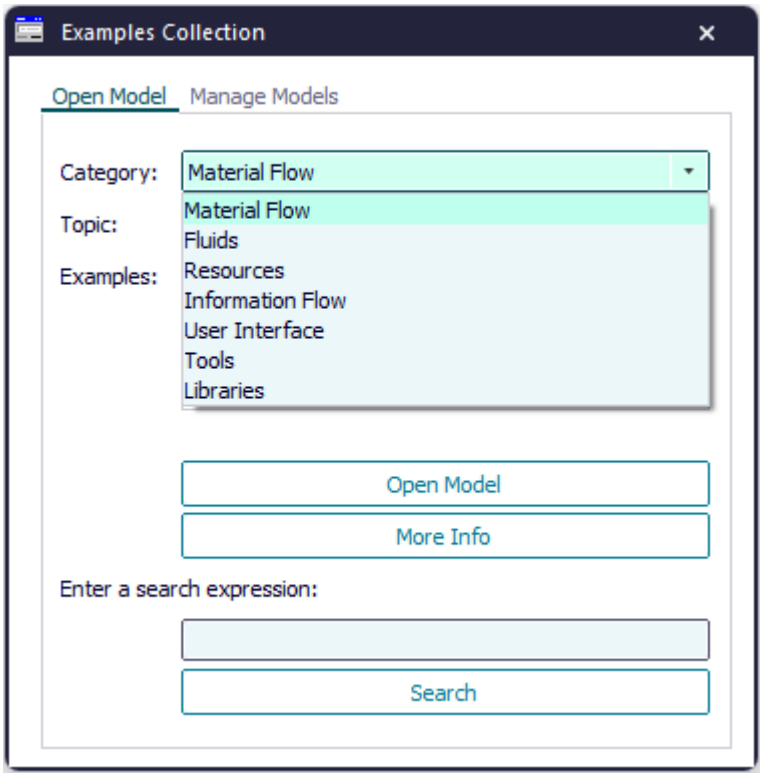
Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



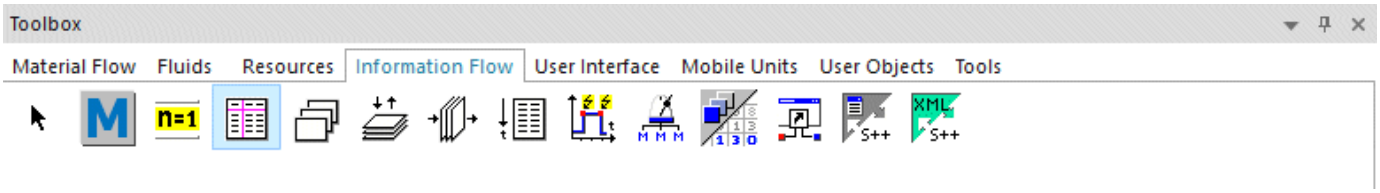
Back to Display Data Stored in an External Program

Working with Lists and Tables

Work with Lists and Tables

Plant Simulation provides several types of lists. They differ in the way they access data they contain. This way you can insert just the right list object to meet your specific modeling needs. You can use lists to provide the material flow objects with data with which they work during the simulation run. You can also write data, such as the results of your simulation runs into lists, export the data, and further process them in other applications.

You can insert a *DataTable*, a *DataList*, a *DataStack*, a *DataQueue*, and a *TimeSequence* into your simulation model from the folder **InformationFlow** in the *Class Library* or from the toolbar **Information Flow** in the *Toolbox*.



- The **DataList** has one column. It accesses the cells randomly by their position. You can add new cells at any position within the *DataList*. When you remove a cell, all cells with a higher number move up one position.
- The **DataQueue** has one column. It accesses the cell you added first: The contents of the first cell you added is the first one to be processed. It adds new cells after the last existing cell.
- The **DataStack** has one column. It accesses the cell you added last. The contents of the last cell you added is the first to be processed. When you add a cell to the top of the stack, all existing cells move one position down. When you remove a cell, the remaining cells each move up one cell.
- The **DataTable** has several columns. It accesses the cells by their column number and their row number. New data you type in overwrites and replaces the existing contents of the cell.
- The **TimeSequence** two columns. It accesses all pairs of cells randomly by their column number and their row number. It adds new entries in ascending order according to the time. Entries with a higher position move up by one position when you remove a previous entry.

The procedures described below are the same for all types of *Plant Simulation* lists, which you insert into your simulation model. Before you can select your own settings, you have to deactivate inheritance: Click  on the **List** ribbon tab so that it is not selected. You can then change settings.

Compare Properties of Lists and Tables

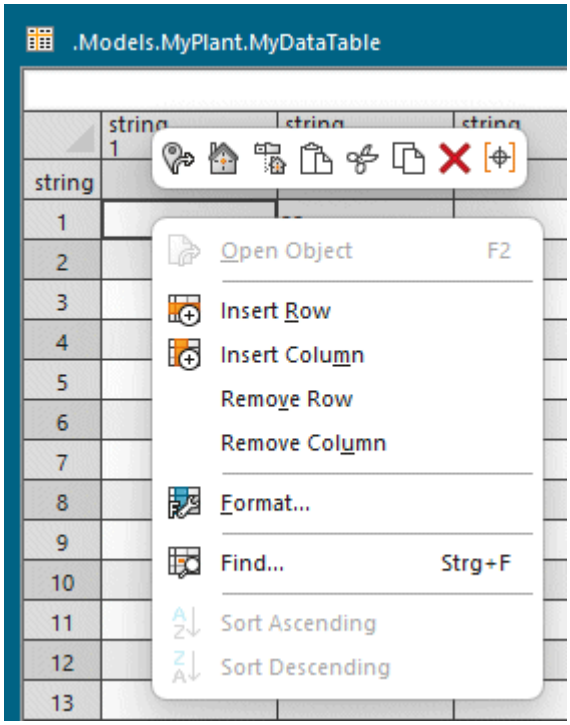
Back to Import Data for the Simulation

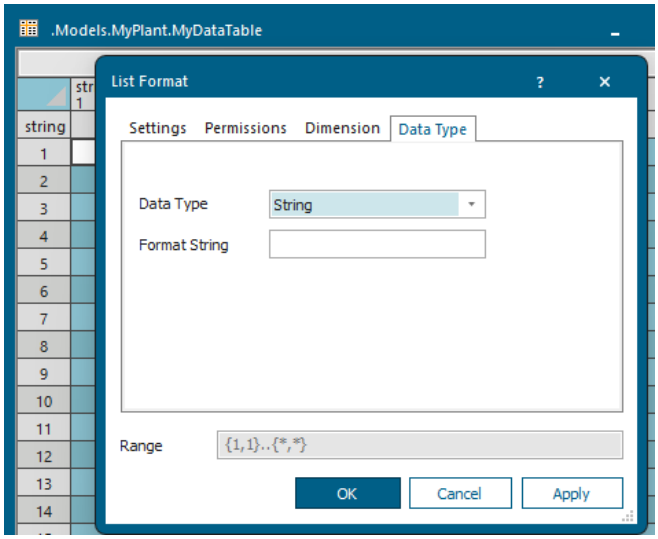
Set the Data Type of a Column

For the *DataList*, the *DataStack*, and the *DataQueue*, which only have one column, you set the data type for the entire list. For the *DataTable* you can select the data type for each individual column or for a range of columns.

To set the data type of a list or table:

- Deactivate inheritance of the format: Click  on the **List** ribbon tab so that the button is not selected.
- Click  **Edit Format** on the **List** ribbon tab. Or
- Click into the column for which you would like to set the data type with the right mouse button and select **Format**.





- To show the tabs **Dimension** and **Data Type**, click in the column header of a column.
- Select one of the data types from the drop-down list.

Data type	Description
Acceleration	applies for the objects **Conveyor**, **Track**, **TwoLaneTrack**, and **Transporter**, meter per second squared
Boolean	true or false
Date	date statement (**dd.MM.yyyy**)
DateTime	date statement, including the time (**dd.MM.yyyy HH:mm:ss**)
Integer	integer value
Length	floating point number, value depends on the unit of the length
List	list with one column, shares properties of the **DataList**
Money	floating point numbers
Object	reference to a simulation model or an object
Queue	list with one column, shares properties of the **DataQueue**
Real	floating point number, such as `3.1415`
Speed	floating point number, value depends on the unit for speed
Stack	list with one column, shares properties of the **DataStack**

Data type	Description
String	characters, numbers and special characters
Table	table with one or more columns, shares properties of the **DataTable**
Time	time statement (hh:mm:ss.ss)
Weight	floating point number, value depends on the unit of the weight

Note

Double-clicking a cell of data type *string* or *boolean* with the value *true* or *false* in a list or table toggles the value from *true* to *false* and vice versa.

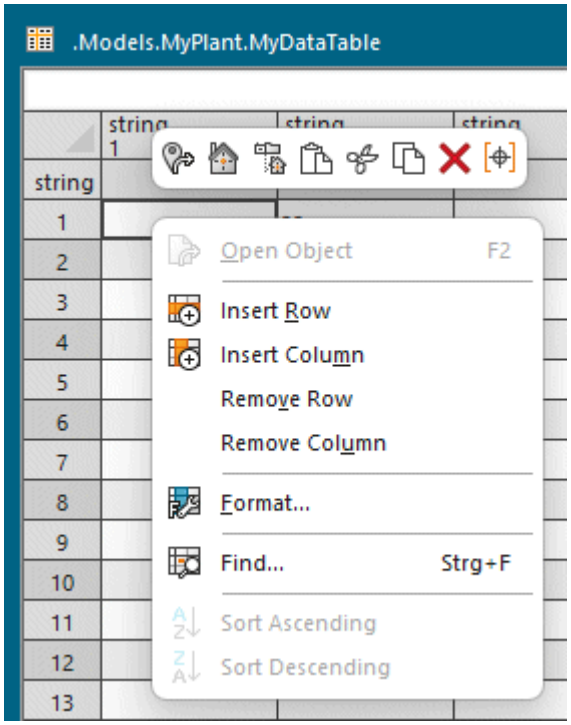
- For the data types **Integer**, **Real**, and **String** you can also enter a **Format String**.
- If you want to hide the data type of the cells in the column, click **Data Type** on the **List** ribbon tab so that the button is not selected.

Back to Tables

Set the Dimension of a List

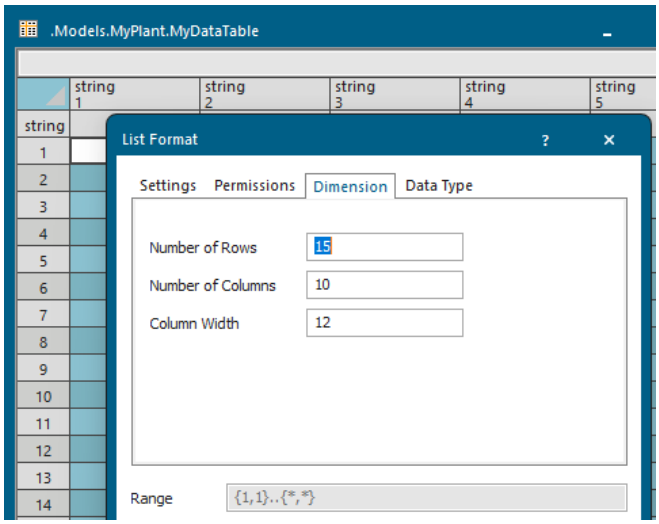
To set the dimension, i.e., the borders of a list or a table:

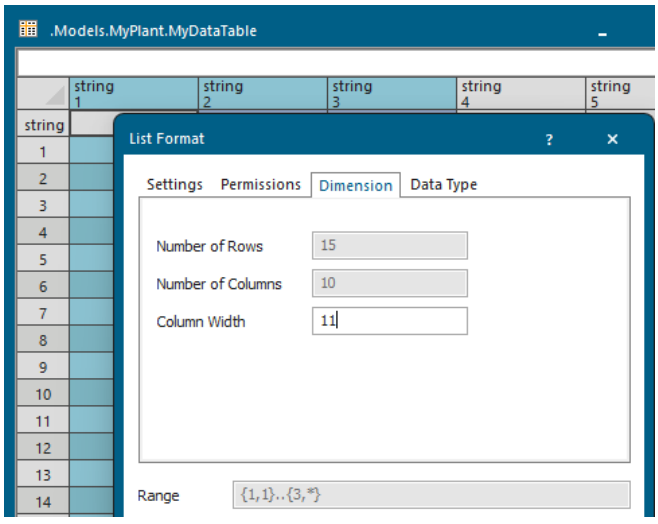
- Deactivate inheritance of the format: Click  on the **List** ribbon tab so that the button is not selected.
- Click  **Format** on the **List** ribbon tab. Or
- Click into the column for which you would like to set the data type with the right mouse button and select **Format**.

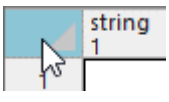


- To set the standard width of a single column, click in the column header. This shows the tabs **Dimension** and **Data Type**. Enter the column width in character widths of a non-proportional font that fit in a cell into the text box **Column Width**.

  To simultaneously change the width of several columns, mark the range of columns before you enter the width. To do so, click in the first column, hold down the mouse button and drag the mouse to the last column. The box **Range** shows which columns you have selected.



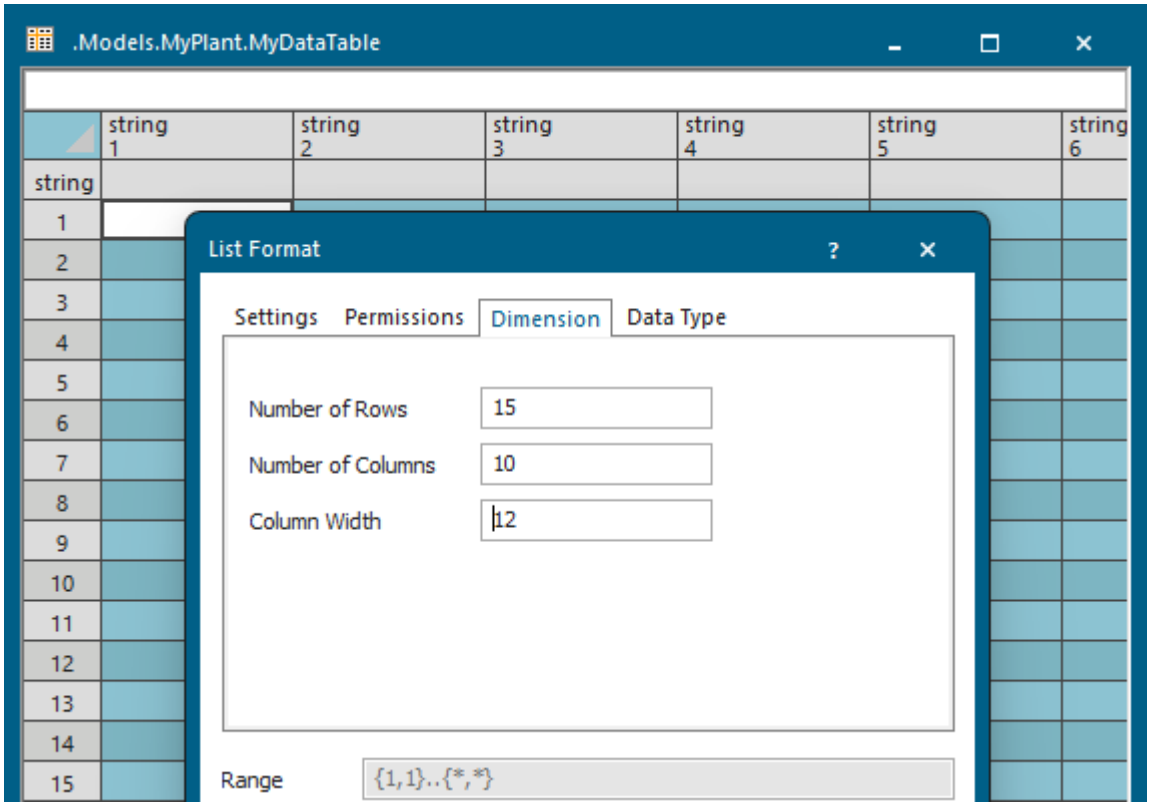


- To limit the size of a table, click **Select All** 

in the top left corner of the table area. Or
- Click the **Select All** button  on the toolbar of the context menu.

For all lists you can enter the **Number of Rows**.

For the *DataTable* you can, in addition, enter the **Number of Columns**. If you do not enter a **Number of Columns** and/or a **Number of Rows**, the size of the list is not limited, which might be memory-consuming.



- To insert a blank column to the left of the selected column, right-click it and select **Insert Column**. *Plant Simulation* assigns the data type **string** to this new column. You can change it afterwards as described above.
- To insert a blank row above the selected row, right-click it and select **Insert Row**.
- To delete the selected column or row, right-click it and select **Cut**.

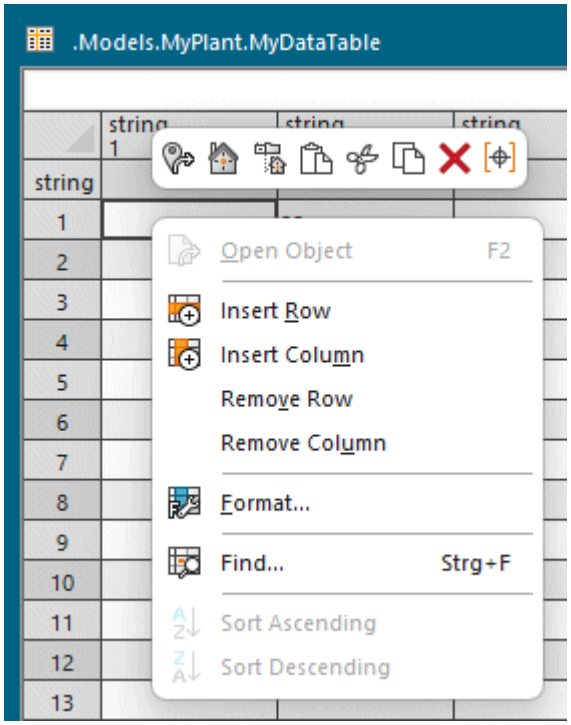
Back to Work with Lists and Tables

Set Alignment and Colors of Cells

To set the alignment, the font size, the font color of cells and the background color of the list or table:

- Deactivate inheritance: Click  on the **List** ribbon tab so that the button is not selected.
- Click  **Format** on the **List** ribbon tab. Or

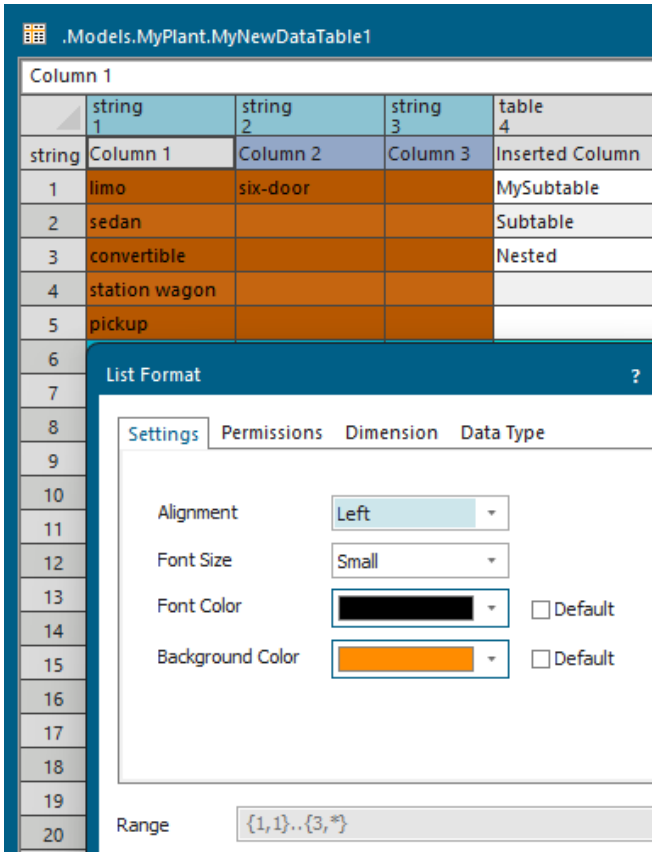
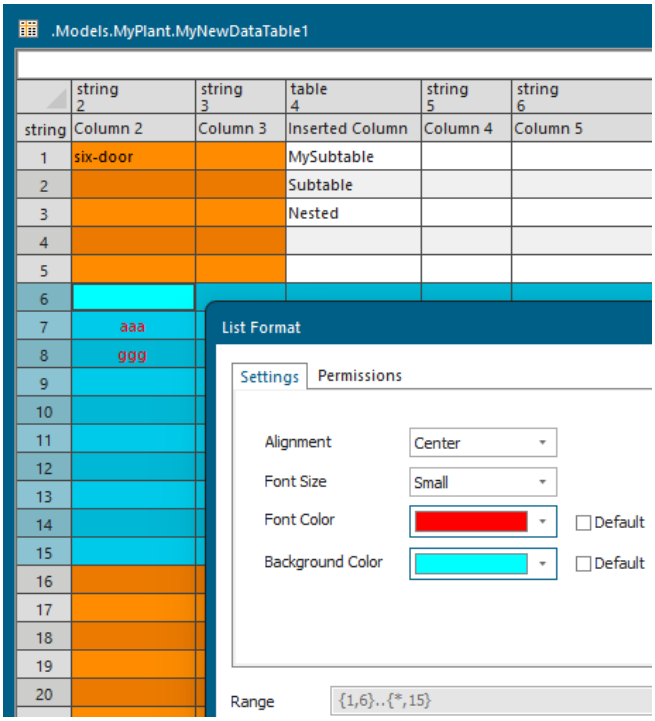
- Click into the column for which you would like to set the data type with the right mouse button and select **Format**.



- Click in the column header and select the columns to which you want to apply the settings. Or

  Click in the row header and select the rows to which you want to apply the settings. The box **Range** show which columns or rows you have selected.

- Select the **Alignment**, the **Font Size**, the **Font Color** and the **Background Color** of the selected columns and rows.

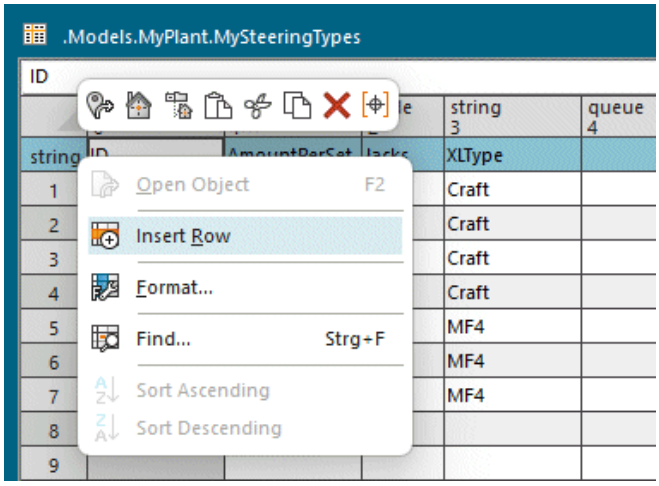
Format Settings Columns	Format Settings Rows
	

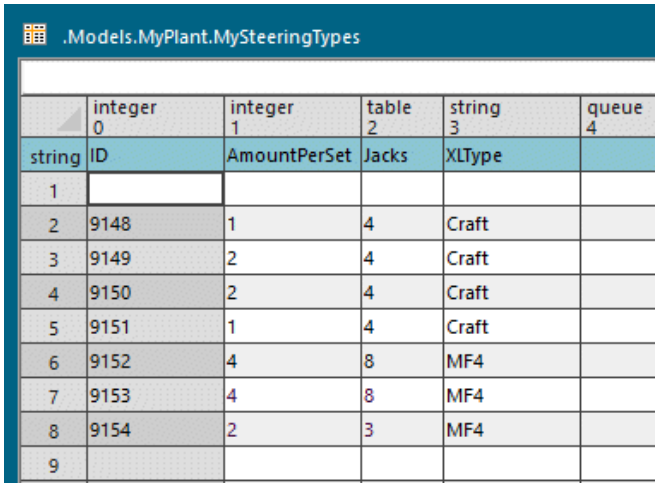
Back to Work with Lists and Tables

Insert, Cut and Delete Rows and Columns

To insert a **row** of empty cells:

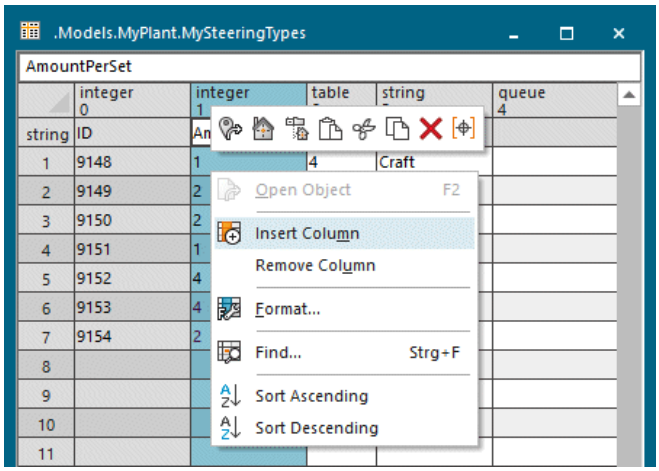
- Right-click into any cell in the row above which you would like to insert a row of blank cells. In our example, we clicked a cell in row 1 to insert a new row above row 1.
- Select **Insert Row**.

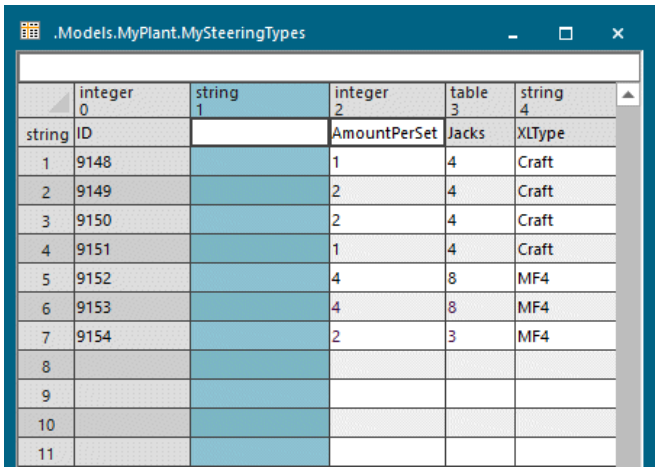




To insert a **column** of empty cells:

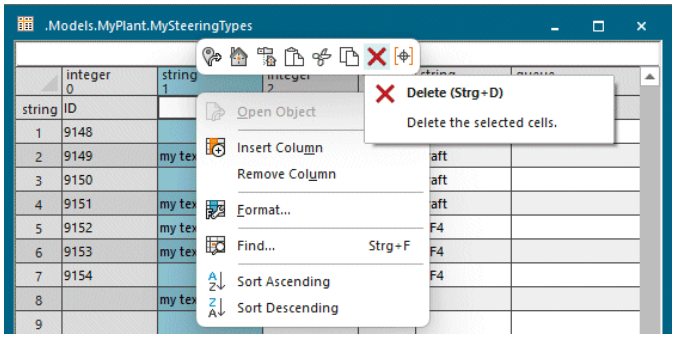
- Right-click into any cell in a column to the left of which you would like to insert a column. If you select the first column, *Plant Simulation* inserts the new column to the right of it! In our example, we clicked a cell in column 1 to insert a new column to the left of column 1. All other columns automatically move one position to the right.
- Select **Insert Column**.

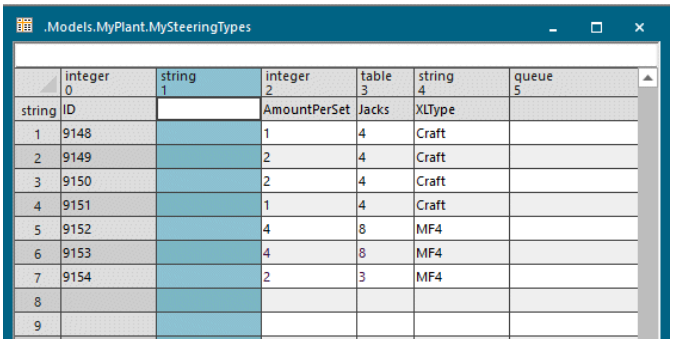




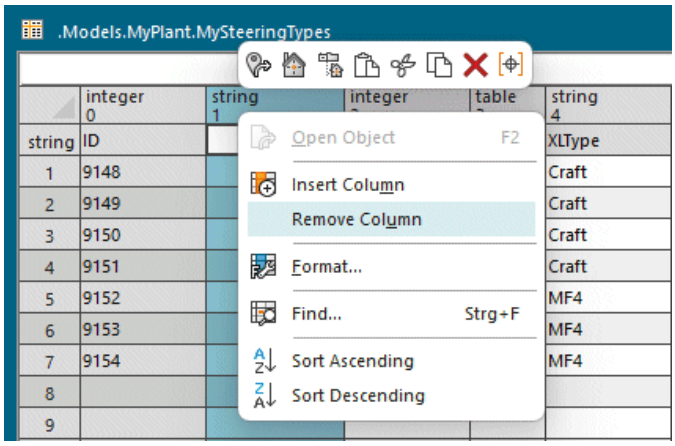
To **delete the contents** of an entire row or of an entire column of cells but to leave the empty row or column in the list, right-click it and select **Delete** on the mini toolbar.

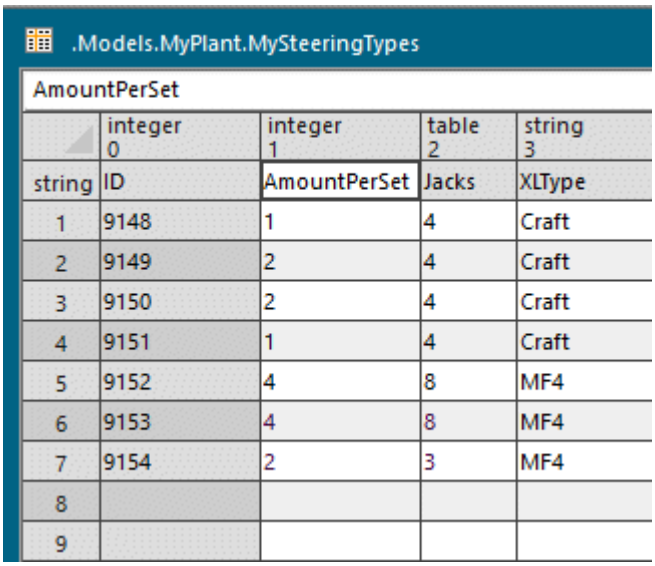
To **clear the contents** of the selected cell, double-click, right-click it and select **Delete** on the context menu.





To **delete an entire row or an entire column of cells with its contents** from the table, right-click in the row header/column header and select **Remove Column**.





Back to Work with Lists and Tables

Work with Data in a List or Table

You can work with a list or table, which you inserted into a *Frame*, like this:

- Click the cell into which you would like to enter data and type in your data. Or type in data into the text box [ ] above the columns and rows.

  When you click in a cell that contains data and start typing, *Plant Simulation* deletes the current contents of the cell and enters the characters you type in.

- You can use the following keys/key combinations to navigate in lists:

  To move one cell down, press **Enter**.

  To move one cell up, press **Shift+Enter**.

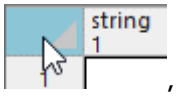
To move one cell to the right, press **Tab**.

To move one cell to the left, press **Shift+Tab**.

To jump to the start of the cell below the current cell, press **Ctrl+Enter**.

To select a range of cells, press **Shift+Cursor Up/Down/Left/Right**.

- To apply the data you entered, press **Enter** or move the cursor into another cell (**Shift**+arrow keys).
- To restore the previous contents of the cell while the cursor is still located in that cell, press **Esc**.
- To move the cursor within the cell, press one of the arrow keys.
- To move from cell to cell to the right or the left within the table, hold down **Shift** and press the right or left arrow.
- To delete the contents of a cell, double-click it, then right-click in the cell and select **Delete**.
- To move or to copy the contents of a cell to another location, use drag-and-drop within the list or table:
  - Click the cell once and drag the mouse over a border of the cell until the cursor changes into crosshairs 

.
  - Press the mouse button, and drag the contents of the cell to **move** it to another location. The move cursor looks like this 
  - Hold down **Ctrl**, press the mouse button, and drag the contents of the cell to **copy** it to another cell. The copy cursor looks like this  .
- To paste data, which you copied to the clipboard with **Home > Copy** into other applications, select **Home > Paste**.
- To scroll up or down in a column, roll the mouse wheel or use the scrollbars.
- To select an entire column or row, their visible and invisible areas, click the column header or the row header.
- To select several contiguous columns of a *DataTable*, click in the column header of the first column, hold down the mouse button and drag the mouse in the column header until you have selected the range you want.
- To select all columns and rows of the list, click **Select All** 

, i.e., the button in the top left corner of the list file, where column and row headers meet. Or click the **Select All** button  on the toolbar of the context menu. Or press **Ctrl+A**.
- To show blank cells in a different color in a list, click **Highlight Empty Cells** on the **List** ribbon tab.
- To create a sublist in a cell of type *Table*, *List*, *Stack* or *Queue*, you can either type in its name and path or use drag-and-drop to enter it.

- To open the sublist or an object, which is contained in a cell of type *Table*, *List*, *Stack*, *Queue* or *Object* bold down **Shift** and double-click the cell. Instead, you can also right-click the cell and select **Open Object** or press **F2**.
- To set the standard column width, select **List > Format > Dimension** > **Column Width** and enter a value. Or you can change it by dragging the cursor:
  - Place the cursor into the topmost row of the column. The cursor changes into a double-headed arrow .
  - Drag the mouse to the left or to the right until the width of the column meets your needs.

Back to Work with Lists and Tables

Work with Data in the DataTable

You can work with a **DataTable**, which you inserted into a *Frame*, like this:

- **Cut or Copy a Range of Cells**

  To cut or to copy a range of cells within a *DataTable*, select the range: Click into the cell that is to be the first corner of the range, hold the mouse button down, and drag the mouse to the opposite corner of the range. *Plant Simulation* highlights the selected range.

  When you cut a range (**Home > Cut**), *Plant Simulation* just removes the contents of these cells, but leaves the empty cells in the table. When you insert the range you cut at a new location, by clicking in the cell that is to be the upper left corner of the range and then select **Home > Paste**, *Plant Simulation* overwrites the contents of the cells in that range. You can also select entire columns or rows: Click in the gray area outside the column or row where the system index is located, i.e., the numbering of the columns and rows. When you select **Home > Cut**, *Plant Simulation* removes the entire column or row. It then shifts the remaining columns to the left, and the remaining rows up. When you select an entire column or row by clicking in the column or row index, *Plant Simulation* pastes a column you cut, to the right of the current column and pastes a row you cut below the current row.

- **Work with Drag-and-Drop in DataTables**

  Accelerator designates the listed key on the keyboard.

To do this	Drag from	To	Accelerator
Move the selected text	table window	table window	—
Copy the selected text	table window	table window	**Ctrl**
Insert selected text	any	table window	any
Copy the selected text	table window	any	**Ctrl**
Cut the selected text	table window	any	—

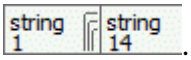
- **Insert a Range of Cells**

  You can paste a range, which you cut or copied from a table, into another range. This only works, if the target range, which you selected, has the same number of columns and rows or is an integer multiple of

the number of columns and rows. If the data types of the copied range are not compatible with the data types of the target range, *Plant Simulation* marks it in red.

- **Hide and Show Columns**

  To hide and then show contiguous columns again:

  - Drag the cursor to the left or right border of the topmost row of the column or the range of columns you would like to hide. The cursor changes into a double arrow ↔.
  - Click the left mouse button and drag to the left until the columns you want to hide are not visible any more. In the example we hid columns 2 through 14 

. You recognize hidden columns by the  symbol that is shown at the end of column 1.

  To show hidden columns again, drag the cursor over the  symbol. The cursor changes into an arrow pointing to the right 

. To expand the columns to their original width, click the left mouse button once.

- **Insert a Drop-down List into a Cell**

  You can use the **Format String** to insert a drop-down list into a cell of a *DataTable*, a *DataList*, a *DataStack*, or a *DataQueue*.

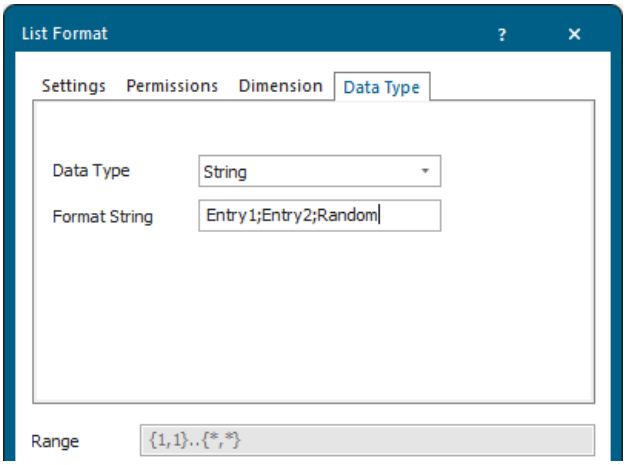
  Select a cell with the data type **String**. Right-click it and select **Format**. Change to the tab **Data Type**.

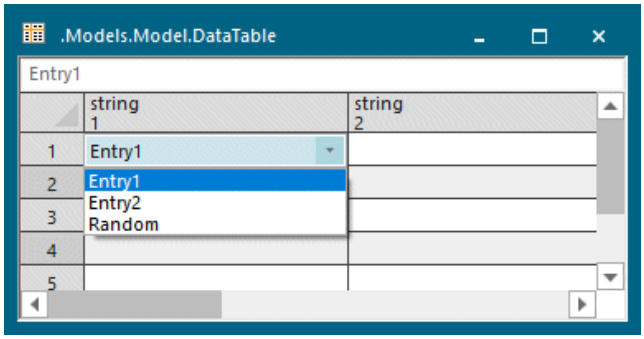
  Enter several terms of your choice, separated by a semicolon, and not followed by a blank into the text box **Format String** to create the drop-down list in the cell of the formatted range into which the user clicks. He can then select one of the terms you entered from the drop-down list.

  If you enter `Entry1;Entry2;Random`, for example, *Plant Simulation* creates a drop-down list with the commands **Entry1**, **Entry2**, and **Random** and provides it in the cells you click within the range you formatted.

  Note

  You have to double-click into the cell(s) where you want to show the drop-down list in the list window to actually show the drop-down list there.





When you open the list window with the method *openDialogBox*, *Plant Simulation* shows the drop-down list immediately.

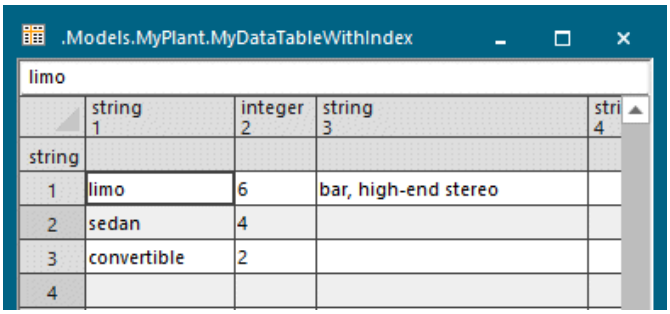
Back to Work with Lists and Tables

Accessing Data in Lists

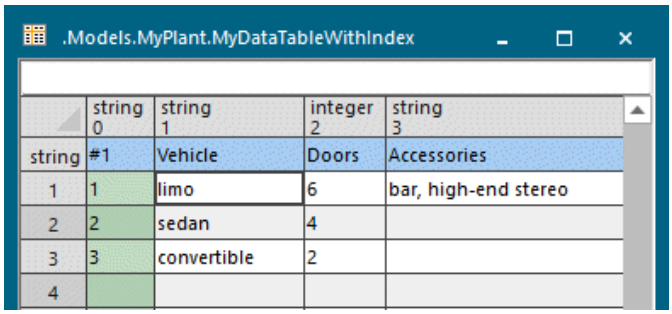
Access Data in Lists

To access a cell in a list or table with a *Method*, you can either use the **system index**, i.e., the **number**, which *Plant Simulation* assigns to columns and rows or you can define your own **user-defined index**, which can be **any meaningful expression**.

System index



User-defined index



The advantage of a user-defined index over the standard system index is that assigning names to columns and rows is more meaningful than the numbers, which *Plant Simulation* assigns by default. The user-defined index `Vehicle["limo",#1]` may tell you and your co-modelers more than the system index `Vehicle[3,1]` when debugging your model. Both expressions access the same cell. As a user-defined index you might, for example, enter:

```
Switch["Light","220 Volts"]
Vehicle["limo",#1]
Plant["Chicago",.building1.drill]
```

In addition, the user-defined index is not as error-prone as the system index: When you add an additional column or a row to the table, *Plant Simulation* increases the system index of the succeeding columns or rows by one. This naturally make any assignment in a *Method* to the previous system index invalid. The identifier of the user-defined index, on the other hand, remains the same and is still valid. Be aware that accessing a user-defined index is slightly slower than accessing the system index.

You can:

- **Set the Column Index**.
- **Set the Row Index**.

- Create a User-defined Column and Row Index.
- Set and Get the Upper Bound of a List.
- Address Columns and Rows with Methods.

Back to Work with Lists and Tables

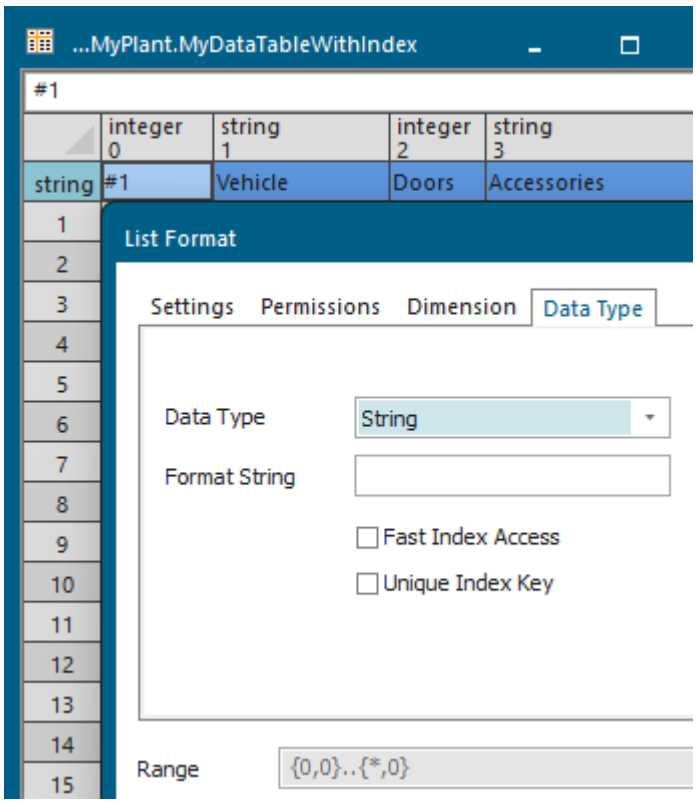
Set the Column Index

To set the column index:

- Click Activate Column Index 

on the **List** ribbon tab.

  When you click the button again, *Plant Simulation* deactivates the column index and deletes the existing contents.
- Select the row of the column index located above the first row of cells and click **Format**.
- Select the data type of the column index from the drop-down list on the tab **Data Type**. For the data types **Integer**, **Real** and **String** you can also enter a Format String to limit the number of digits the user can manually enter into these cells.



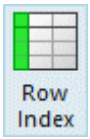
- To allow for quickly accessing the user-defined column index, select the check box Fast Index Access.

- To only allow unique entries in user-defined indexes, depending on the data type, select the check box **Unique Index Key**.

Back to Access Data in Lists

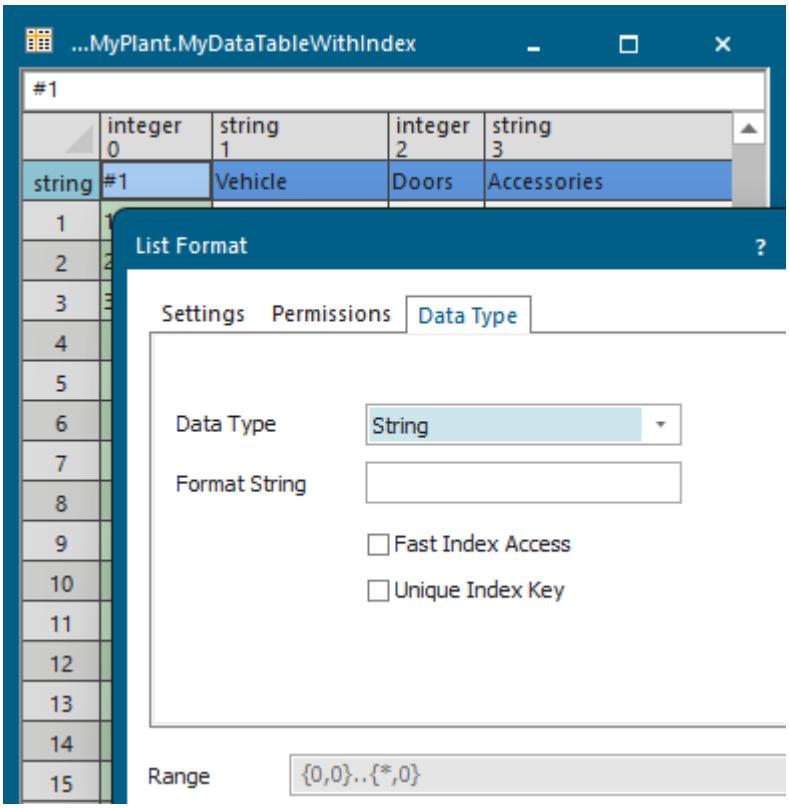
Set the Row Index

To set the row index:

- Click **Activate Row Index** 

on the **List** ribbon tab.

  When you click the button again, *Plant Simulation* deactivates the row index and deletes the existing contents.
- Select the column of the row index and select the command **Format**. Select the data type of the row index from the drop-down list on the tab **Data Type**. For the data types **Integer**, **Real** and **String** you can also enter a **Format String**.
- To allow for quickly accessing the user-defined row index, select the check box **Fast Index Access**.
- To only allow unique entries in user-defined indexes, depending on the data type, select the check box **Unique Index Key**.



- To allow for quickly accessing the user-defined column index, select the check box **Fast Index Access**.

- To only allow unique entries in user-defined indexes, depending on the data type, select the check box **Unique Index Key**.

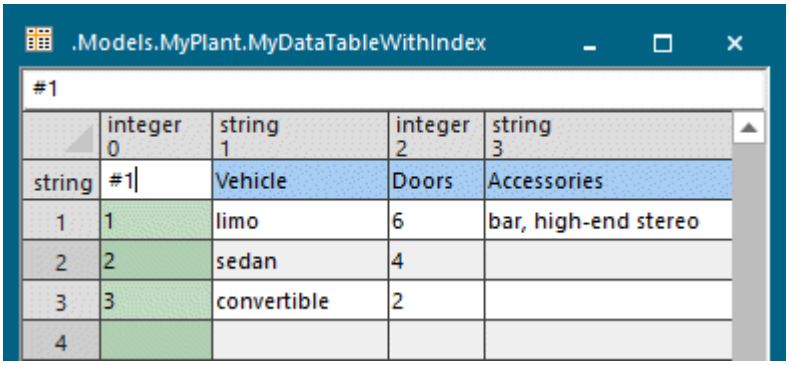
Back to Access Data in Lists

Create a User-defined Column and Row Index

To create a user-defined column index and a user-defined row index:

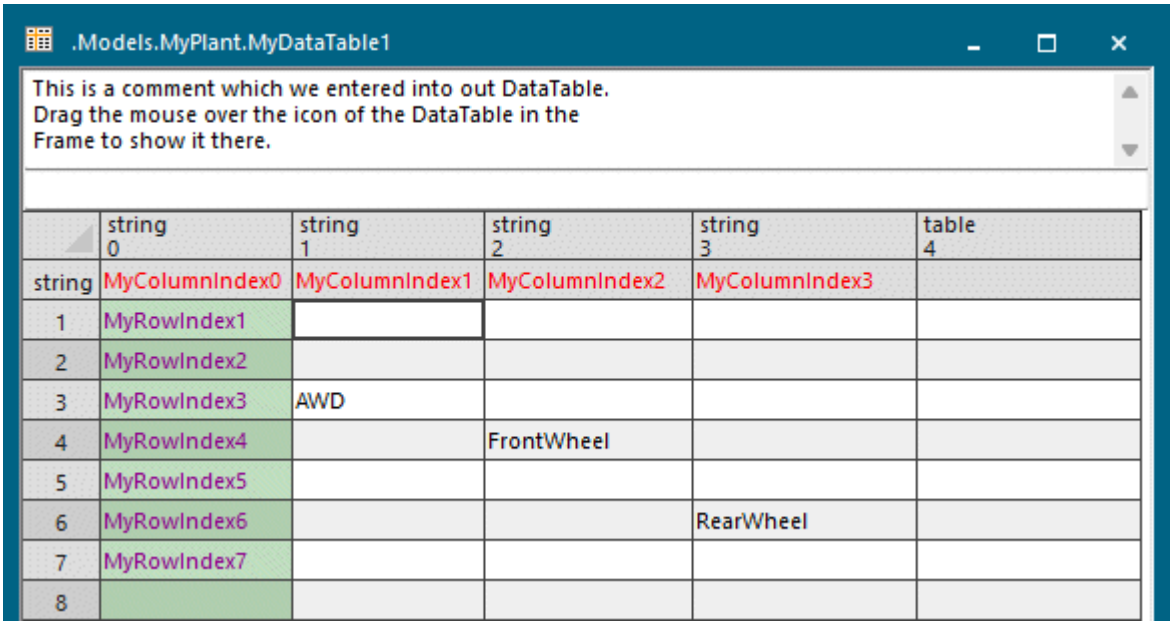
- Deactivate inheritance: Click  in the **List** ribbon tab so that the button is not selected.
- To activate and to show the column index, click **Activate Column Index**. Enter a meaningful term into the first row of cells to the right of **string**.

  In most cases, you will select the data type **String** for the user-defined column index. When you select **Integer**, you have to enter a number sign **#** in front of the term you enter as your user-defined index to distinguish it from the system index.



- To activate and to show the row index, click **Activate Row Index**. Enter a meaningful expression into the first column of cells below **string 0**.

  In most cases, you will select the data type **String** for the user-defined row index. When you select **Integer**, you have to enter a number sign **#** in front of the expression, which you enter as your user-defined index, to distinguish it from the system index.



	string 0	string 1	string 2	string 3	table 4
string	MyColumnIndex0	MyColumnIndex1	MyColumnIndex2	MyColumnIndex3	
1	MyRowIndex1				
2	MyRowIndex2				
3	MyRowIndex3	AWD			
4	MyRowIndex4		FrontWheel		
5	MyRowIndex5				
6	MyRowIndex6			RearWheel	
7	MyRowIndex7				
8					

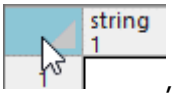
Note

When both column and row index are active, the cell [0,0], i.e., the cell at which column and row index intersect, counts as part of the **column** index, not as part of the row index.

Back to Access Data in Lists

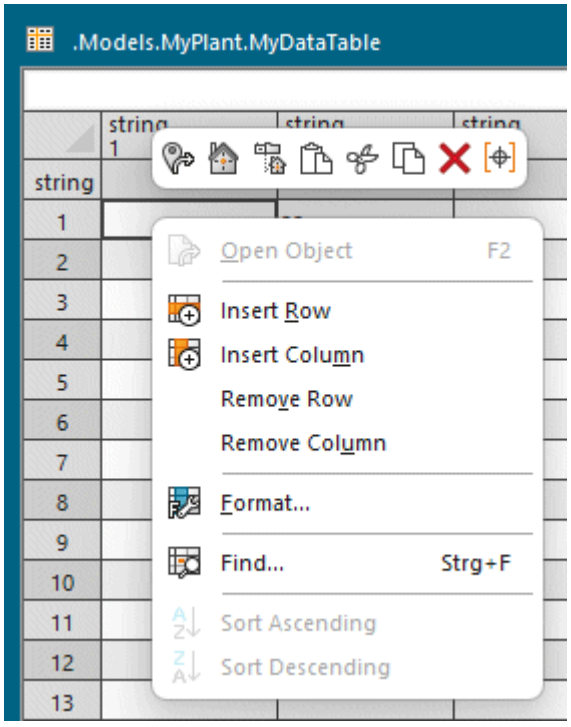
Set and Get the Upper Bound of a List

To set the upper bound of a list or table:

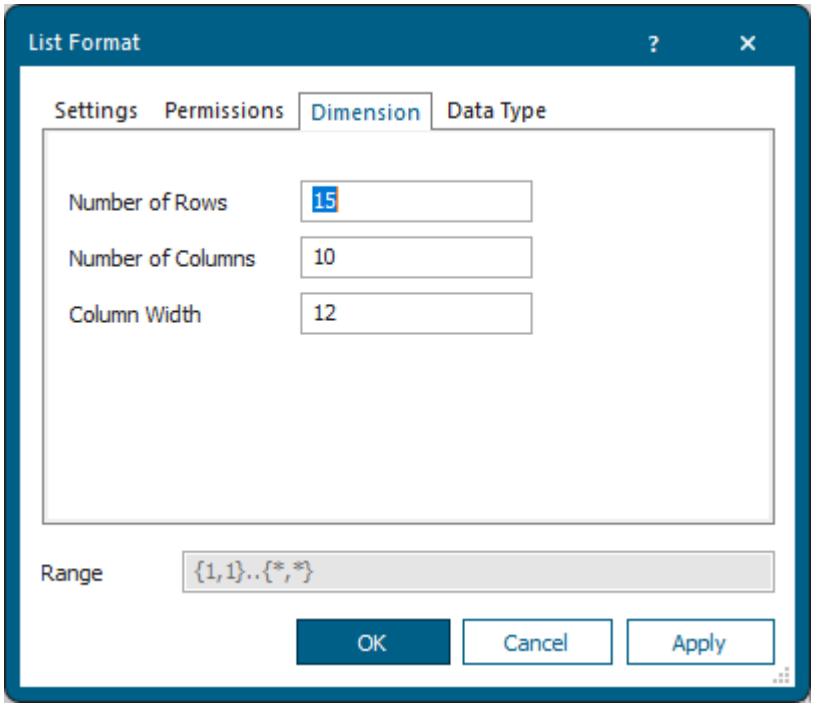
- Deactivate inheritance: Click  on the **List** ribbon tab so that the button is not selected.
- To select the entire list or table, click **Select All** 

, i.e., the button in the top left corner of the list file, where column and row headers meet. Or click the **Select All** button  on the toolbar of the context menu. Or press **Ctrl+A**.
- Click  **Format** on the **List** ribbon tab. Or

  Click into the column and select **Format**.



- Click the **Tab Dimension**. Enter the **Number of Columns** and the **Number of Rows**. *Plant Simulation* automatically sets the lower bound to 1. When you use a user-defined index the lower bound is 0.

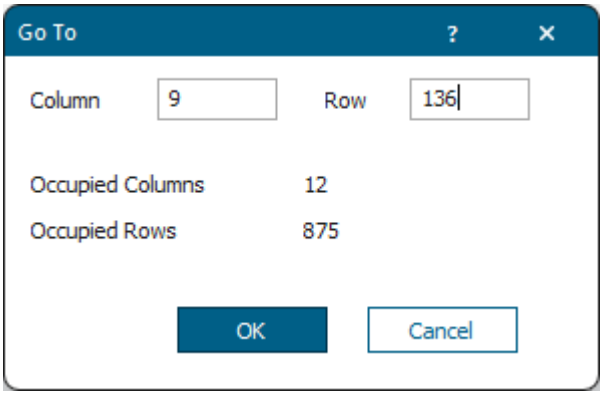


For lists with one column, you can also set the upper bound with the attribute **MaxDim**. For lists with two columns you can set it with the attributes **MaxXDim** and **MaxYDim**. For tables the read-only attributes **XDim** and **YDim** return the dimension of the current occupancy.

The read-only attribute **XDimIndex** returns the last cell of the column index, which contains an entry. The read-only attribute **YDimIndex** returns the last cell of the row index, which contains an entry.

The dialog **Go To** also shows the number of occupied columns and rows.

To move to a certain cell within a list or table, click **Go To** on the **List** ribbon tab and type in the location of the cell.



Back to Lists

Address Columns and Rows with Methods

Plant Simulation uses these conventions when addressing columns and rows in a *DataTable* in *Methods*.

You can:

Set the Format of Columns and Rows

- The syntax for setting the format of columns and rows is:

```
setXX(Parameter:any, ..., Parameter:any, Parameter:integer)
```

- The number of parameters always is greater than or equal to 2.
- The last parameter sets—depending on the action you want to perform—the column or row:

  You can use the system index, i.e., the absolute number of the column or row. Or

  You can use a user-defined column or row index. To set a common format for a contiguous range of columns or rows, first define a range. For column functions it only applies to the columns within the specified range. Enter an asterisk (*) to designate all rows within the specified range. For row functions it only applies to the rows within the specified range.

  Compare this example:

```
MyDataTable.setDataType({3,*}..{4,*},6,"column1","real")
```

Enter	to designate
`{3,*}..{4,*}`	the range, all cells in columns 3 and 4 in this case
6	the system index of a column
`"column1"`	a column index
`"real"`	a value

Get the Format of Columns or Rows

- The syntax for getting the format of columns and rows is:

```
getXX(ColumnOrRow:any)
```

- The number of parameters always is 1.
- The parameter *ColumnOrRow* of data type *any* designates the column or the row:
  - You can use the system index, i.e., the absolute number of the column or row. Or
  - You can use a user-defined row or a user-defined column index.

    Compare this example:

    ```
    MyDataTable.getDataType(1)
    ```

    This *Method* returns the data type of column 1.

Back to Access Data in Lists

Search Lists with Methods

Lists and tables have an internal cursor pointing to a cell of the list. You can query the current position of the cursor with the attribute **Cursor** and set it anew by assigning an integer value to it. A search, for example with the method **find**, begins at the current cursor position. Before starting the search, make sure that the list cursor is placed at the cell from which on you want to search.

After a successful search, *Plant Simulation* places the cursor in the cell where it found the value. If the list contains the value you searched for more than once, a new call of the method *find* finds the next value starting at that cell, and so on.

When you set a range for the *find* command, *Plant Simulation* determines the starting position of the cursor by combining the cursor and the range. The search begins at the first entry of the list in the range you entered, starting at the cursor position. The search ignores any range, which is located before the current cursor position. If the cursor is located in the middle of the range you entered, *Plant Simulation* does not search for entries before of the cursor position! The search goes on until *Plant Simulation* finds the value or reaches the end of the list. When *Plant Simulation* finds the value you searched for, it places

the cursor into the cell, which contains the value. A new search begins at the next cell. If the search does not find the value, the internal cursor remains in the cell it had before the search.

```
MyDataStack.find(12.34)
// finds the floating point value 12.34 starting from the current cursor
position
MyDataQueue.find(42)
// finds the integer value 42 starting from the current cursor position
MyDataList.find({1},{4}..{8},"drill")
//finds the string "drill" in row 1 and in rows 4 to 8 starting from the
current cursor position
MyDataTable.find("a")
// finds the string "a" in the entire table starting from the current
cursor position
MyDataTable.setCursor(3,1)
// set cursor so that next find starts search from beginning
MyDataTable.find({3,*},"a")
// finds the string "a" in column 3 starting from the current cursor
position
MyDataTable.find({1,1}..{4,*},"a")
// finds the string "a" in columns 1 to 4 in all rows starting from the
current cursor position
```

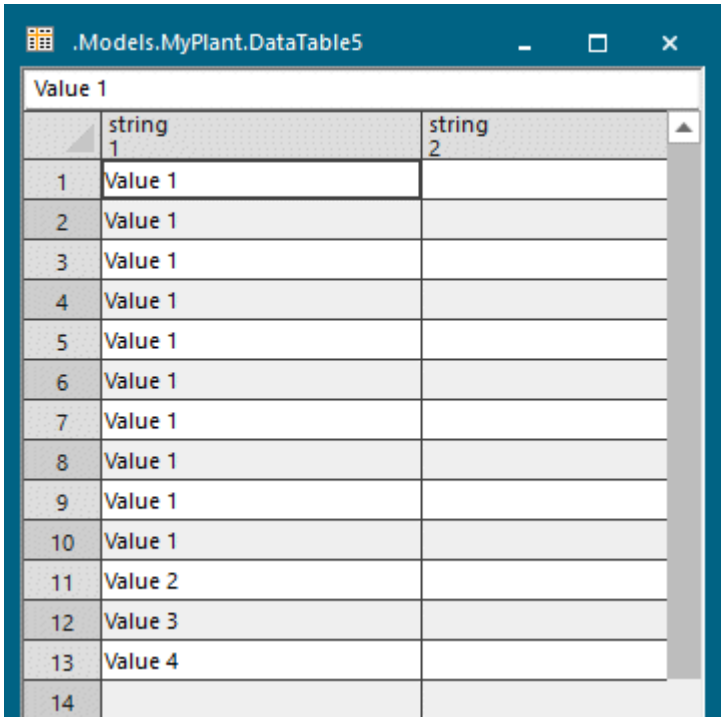
```
var MyDataTable.setCursor(1,1)
// starts searching for "abc" from the beginning to the end
if MyDataTable.find("abc")
   print MyDataTable.CursorX, " ",MyDataTable.CursorY
else
   print "not found"
end
```

The same principle applies to tables. As a table has rows and columns, it naturally has two cursors: **CursorX** designates the column and **CursorY** designates the row, which identifies the cell.

Note

Set the cursor anew after inserting or deleting rows, to determine its position.

In the example below we have this *DataTable* containing several values in a column.



We need to know if one or several of these values already exist in the *DataTable*. We program this as follows in a *Method*.

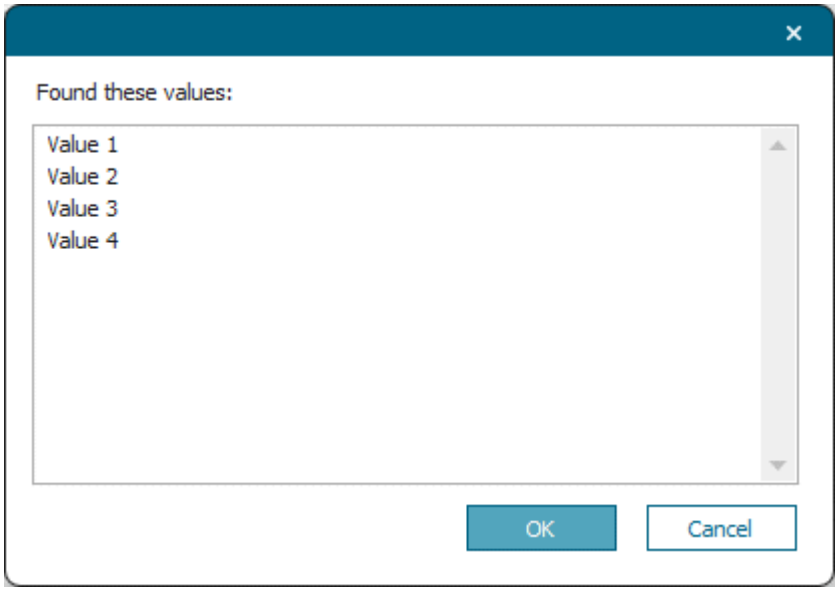
```
var lst : list; lst.create

for var i := 1 to DataTable5.YDim
   lst.setcursor(1)
   // Sets the cursor into the first cell. Searching starts there.

   if not lst.find(DataTable5[1, i])
   // Checks if a value already exists in the list variable.
      lst.append(DataTable5[1, i])
   end
next

promptListN(lst, "Found these values:")
// Presents all values in the list.
```

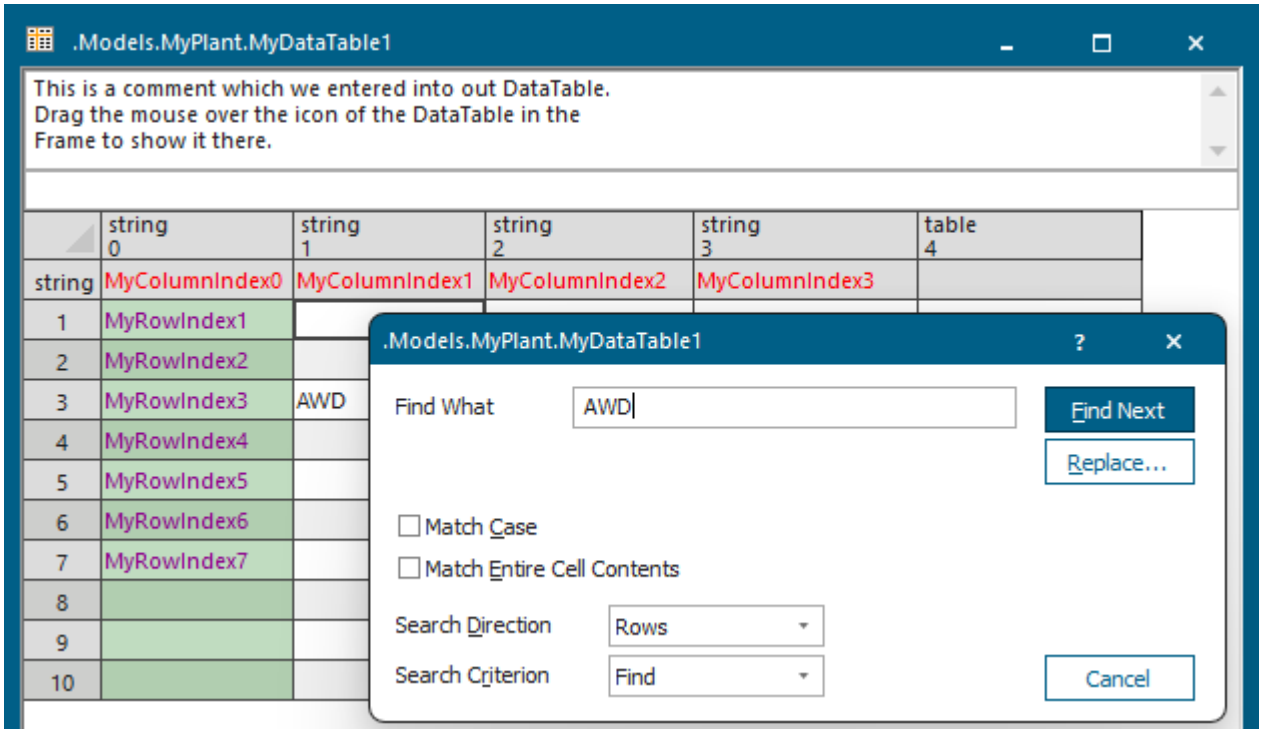
We present already existing values in a dialog box with the method **promptListN**.



Back to Work with Lists and Tables

Search Lists with the Find Dialog

To search and optionally replace a string with another string, right-click in the list and select **Find** or press **Ctrl+F**.



- Type the search term you would like to find into the text box **Find What**.

- Select **Match Case** to only find text that has the same pattern of upper and lower case as the search term you typed into the text box **Find What**.
- Select **Match Entire Cell Contents** to only find the characters in cells, which exactly and completely match the search term you typed into the text box **Find What**.
- Select if you want to **Search in Rows**, i.e., to the right across rows, or down through **Columns**.
- Select the **Search Criterion** from the drop-down list:

  Find: Finds the search term you typed into the text box **Find What**, compare the method **find**.

  Find ceil(ing): Finds a value greater than or equal to the search term you typed into the text box **Find What**, compare the method **findCeil**.

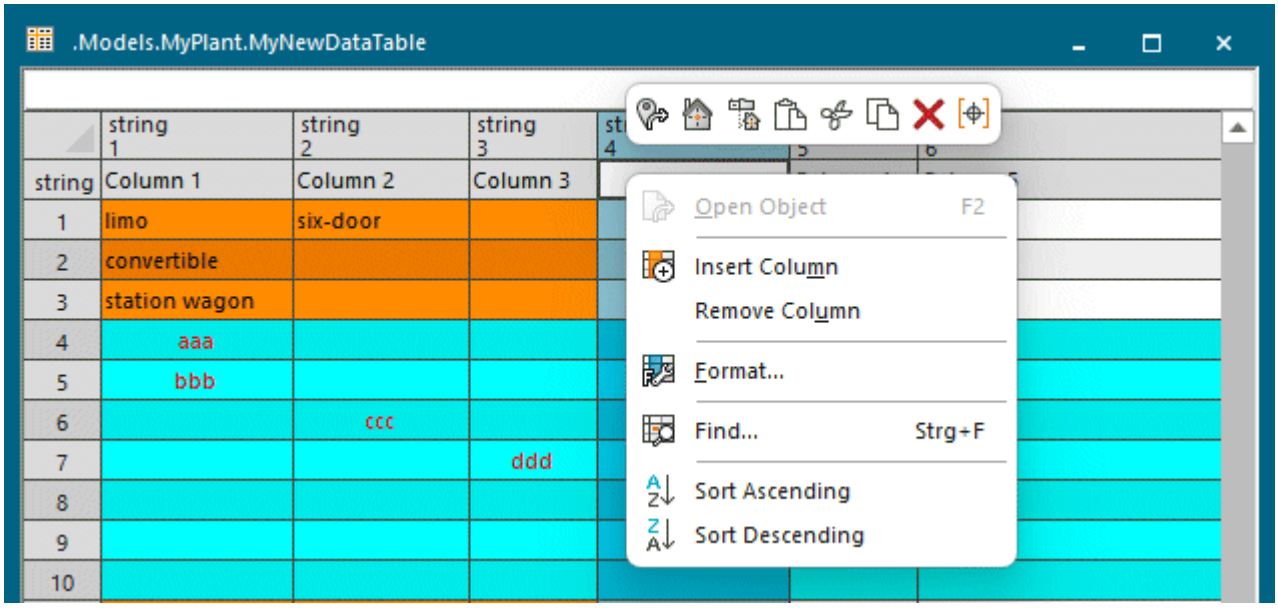
  Find floor: Finds a value less than or equal to the search term you typed into the text box **Find What**, compare the method **findFloor**.
- Click **Find Next** to find the next instance of the search term, which you type into the text box **Find What**.
- Click **Replace** to show the text box **Replace With**. Type the search term into the text box **Replace With** that is to replace the search term you typed into the text box **Find What**.
- Click **Replace** to replace the search term.

Back to Work with Lists and Tables

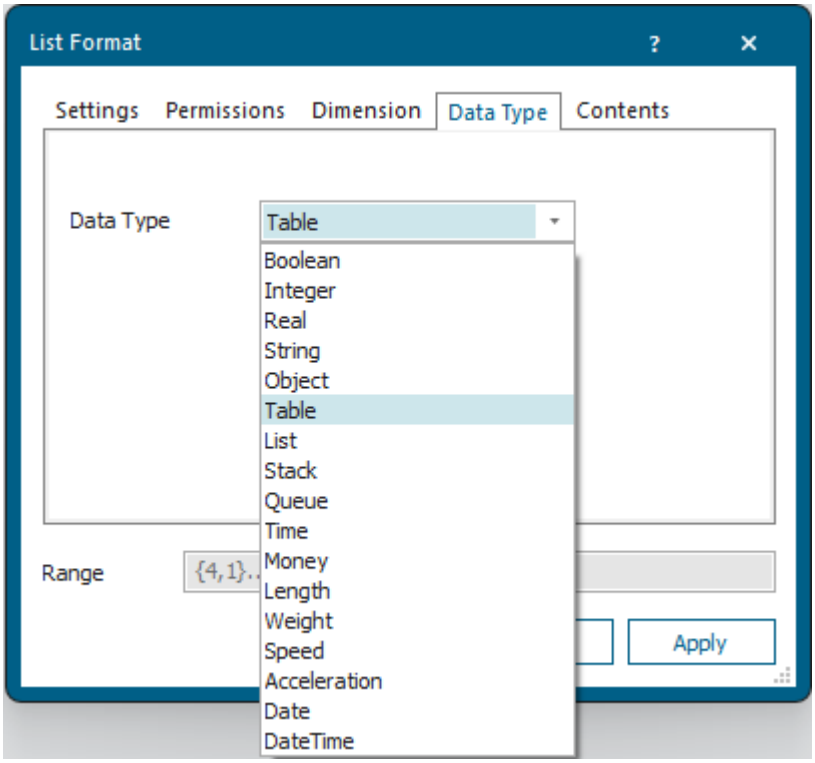
Create Lists within Lists and Tables

To **create a sublist** or a **subtable** in a cell of the *DataList*, the *DataStack*, the *DataQueue*, or the *DataTable*:

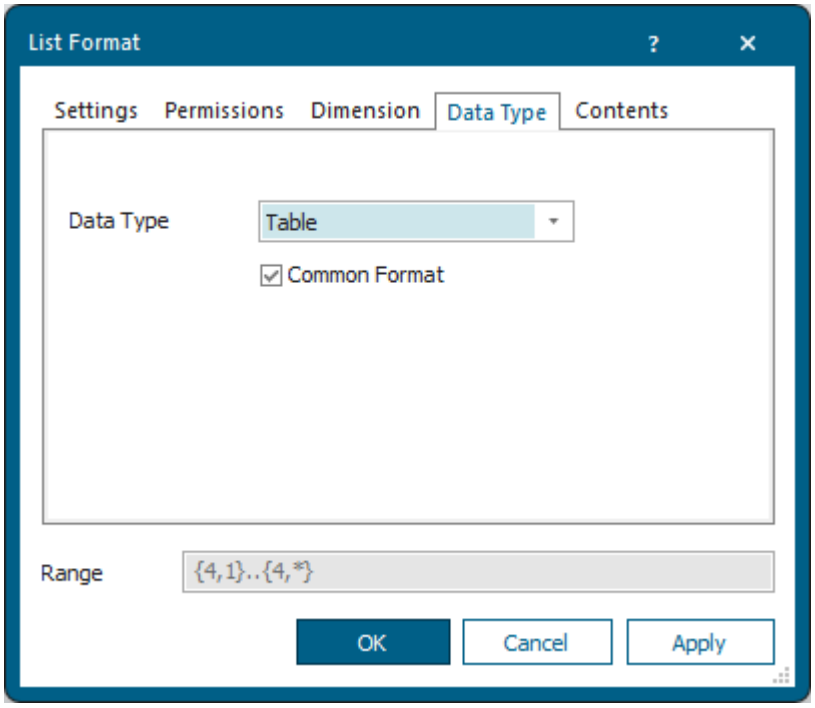
- Open the list object to which you want to add a sublist/subtable.
- Click  on the **List** ribbon tab so that the button is not selected.
- Right-click the column header of the column to which you want to add a sublist and select **Format**.



- Select the data type of the sublist you want to insert: **Table**, **List**, **Stack**, or **Queue**. You will choose the data type depending on how you want to access the entries of the sublist.

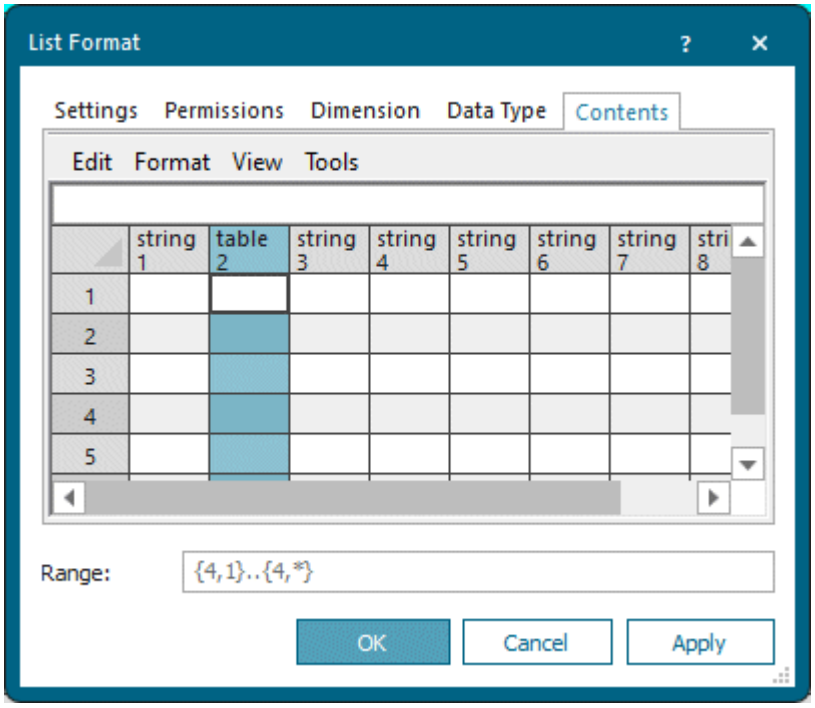


If you want all sublists in that column to have the same formatting, select **Common Format**.



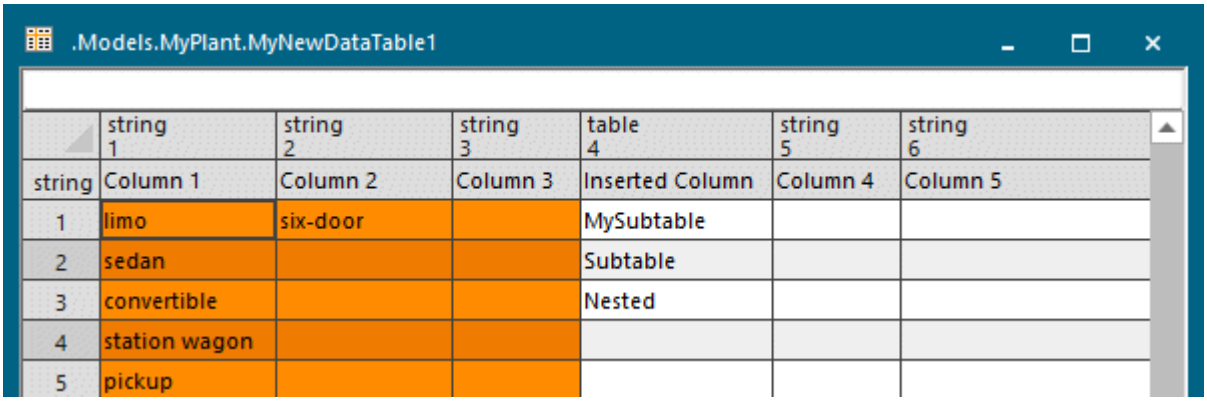
Apply formatting properties on the **Tab Contents**. Click **OK**.

Note
You can also accomplish this with the method **setCommonFormat**.



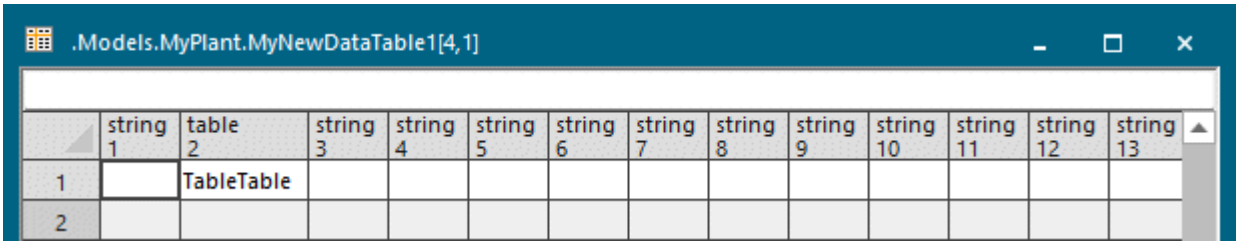
- Click **OK**.

- Enter a name of your choice, which identifies the sublist, into the cells of the changed column. This name identifies the subtable.



	string 1	string 2	string 3	table 4	string 5	string 6
string	Column 1	Column 2	Column 3	Inserted Column	Column 4	Column 5
1	limo	six-door		MySubtable		
2	sedan			Subtable		
3	convertible			Nested		
4	station wagon					
5	pickup					

- Hold down **Shift** and double-click the *subtable*, **MySubtable** in our case, to open it. Edit the *subtable* according to your needs.

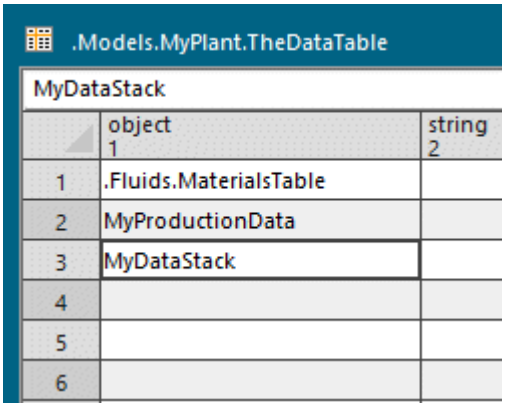


	string 1	table 2	string 3	string 4	string 5	string 6	string 7	string 8	string 9	string 10	string 11	string 12	string 13
1		TableTable											
2													

To **insert a list object** from a *Frame* or the *Class Library* into a cell of the list objects *DataList*, *DataStack*, *DataQueue*, or *DataTable*:

- Open the list object to which you want to add a sublist/subtable.
- Click  on the **List** ribbon tab so that the button is not selected.
- Right-click the column header of the column to which you want to add a sublist and select **Format**.
- Select the data type **Object**.
- **Drag the table** to a cell of the list and drop it there. This inserts the **absolute path** of the inserted list.

  Type the name of the list you want to insert into the cell, when this list is located in the same *Frame*. This way *Plant Simulation* uses the **relative path**.



To open the sublist or a list object, which is contained in a cell of type *Table*, *List*, *Stack*, or *Queue*, hold down **Shift** and double-click into the cell. Instead, you can also right-click the cell and select **Open Object** or press **F2**.

Compare Accessing the Name of a Sublist with a Method

Back to Work with Lists and Tables

Sort DataList, DataTable, and TimeSequence

To sort the contents of a *DataList*, a *DataTable*, or a *TimeSequence*:

- To sort the selected column in ascending (A to Z, or 0 to 9) order, click **Sort Ascending [lists]** or **Sort Ascending** on the context menu.
- To sort the selected column in descending order (Z to A, or 9 to 0), click **Sort Ascending [lists]** or **Sort Descending** on the context menu.

You can also only sort the *DataList*, the *DataTable* and the *TimeSequence* with *methods*:

- The method **sort** sorts the lists in ascending (A to Z, or 0 to 9) or in descending order (Z to A, or 9 to 0).
- The method **inOrder** sorts a value into an existing sequence of values at the position where you want it to be.

Back to Work with Lists and Tables

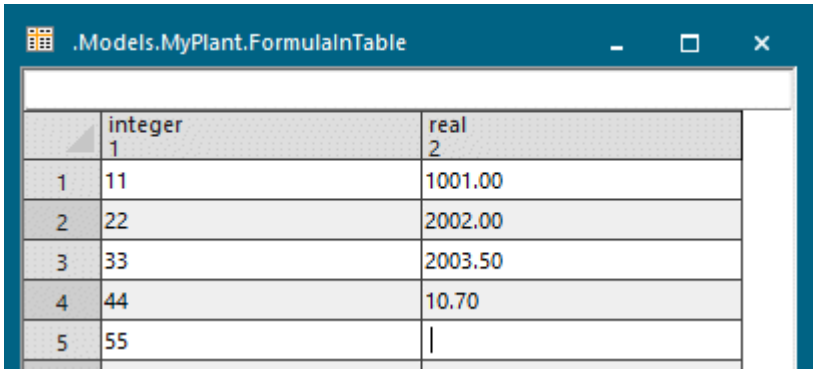
Calculate Values with a Formula

A **Formula** is an expression, which can read out and link the values of other table cells and attribute values of objects. With these values, the formula then makes the calculations you specified.

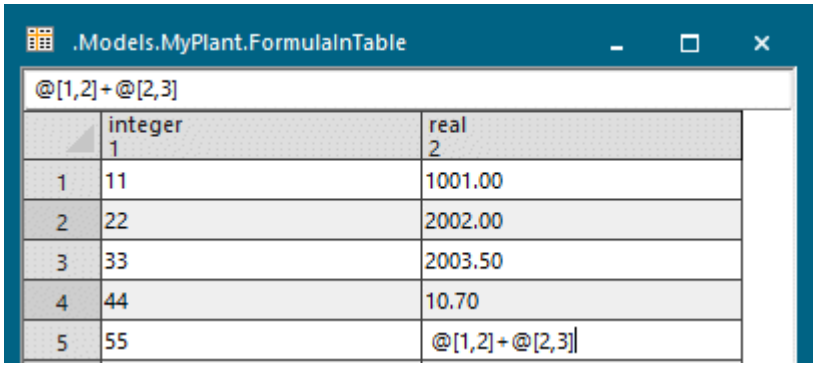
For linking the values within a formula, you can use the same arithmetic operators and mathematical functions as you use in *methods*, compare **Operators and Expressions**.

Procedure

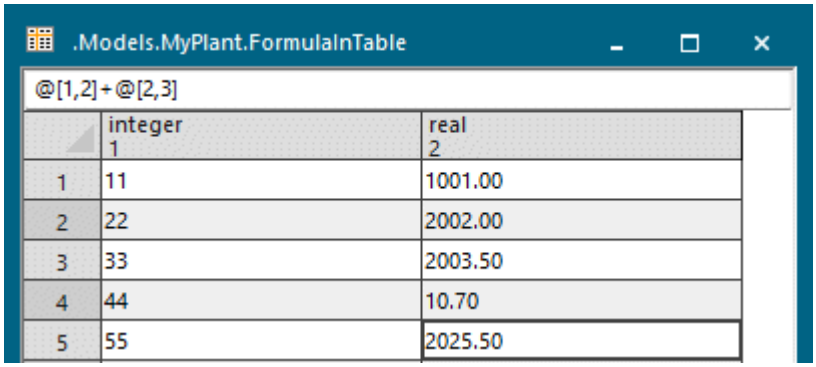
1. To activate *Formula* mode, click **Create Formula** on the **List** ribbon tab so that it is selected  .



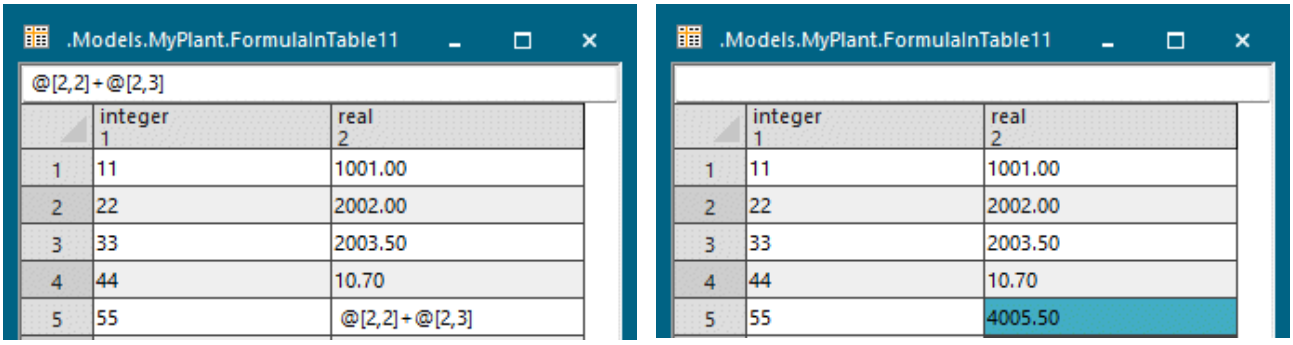
2. To enter a formula into a cell, click in it and type the expression in. Instead, you can also type it into the text box above the list. In our example we add the value of cell 2 in column 1 to the value of cell 3 in column 2. To do so, we typed in `@[1,2]+@[2,3]`.



3. To show the result of the calculation of the formula, i.e., its value, in the cell, press **Enter**.



4. To show the formula itself in the text box, double-click in the cell that contains the formula. Then, you can change the formula. You can also access the values in a column via the column index.



The list shows cells containing a formula in color. Turquoise designates a formula with a correct syntax, red a formula with syntax errors.

Access a Cell in a DataTable with @ in a Formula

Within a formula you can access the value of another cell of the same *DataTable* with the anonymous identifier **@**:

Formula	Executes
`@[1,1]+@[1,2]`	adds the contents of the cell [1,1] to the contents of the cell [1,2].
`@[1,1]*track.length`	multiplies the contents of cell [1,1] with the length of the object track.
`@[1,@.ydim]+5`	adds 5 to the value of the last cell in the first column.
`@[xSelf+1,ySelf]-7`	subtracts 7 from the value of the neighboring cell to the right
`@.sum({3,*})`	computes the sum of the third column.
`@.min({1,2}..{1,*})`	determines the smallest value of the first column, starting from cell 2.

Note

The data type of the result of a formula has to have the same data type as the cell or the column in which the cell containing the formula is located.

Access a Cell in a Local Variable of Data Type Table with ? in a Formula

Within a formula you can access the value of another cell of the same local variable of data type *table* with the anonymous identifier **?**

Access a Sublist with ?

In sublists, you can access the list, into which you inserted the sublist, with the anonymous identifier **?**. For user-defined attributes of lists, the anonymous identifier **?** accesses the object for which you defined the user-defined attribute.

`xSelf` and `ySelf` contain the number of the column or the number of the row respectively, which contains the formula. This way you can easily access neighboring cells.

Catch Runtime Errors in Formulas

You can catch runtime errors in formulas with local error handlers.

To catch an error in a table formula for example, add a user-defined attribute of data type *Method* to the *DataTable* and name it `ErrorHandler`. Assign an empty string `("")` to the error message to suppress the error. If the error is suppressed, you can assign a new value to the wrong table cell by letting the error handler return a value.

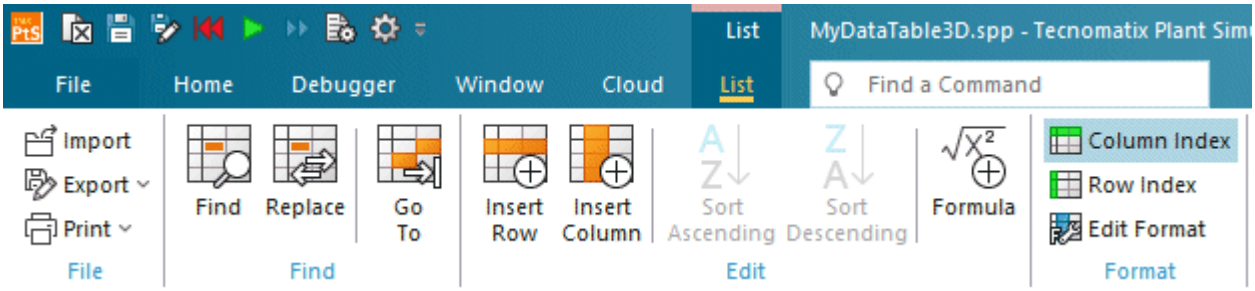
Back to Work with Lists and Tables

Import or Export the Contents of a List

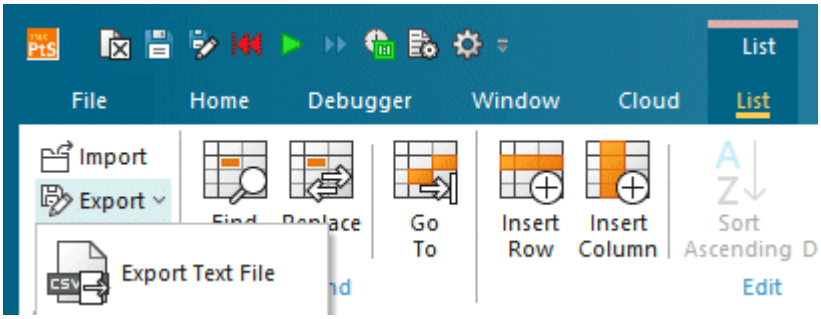
You can save an *Plant Simulation* list in several formats:

- To save the list with all of its *Plant Simulation* formatting as a *Plant Simulation* list, click **Export > Export Object File** on the **List** ribbon tab. You can then open the saved list object within a list in other *Frames* or in other simulation models.

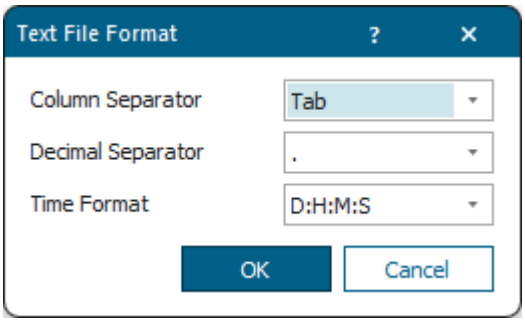
  To import a *DataTable*, which you saved as an .psobj file into another *DataTable*, click **Import** on the **List** ribbon tab. To import a list with one column into a *DataTable*, select the same data type for the left column that the list with one column has.



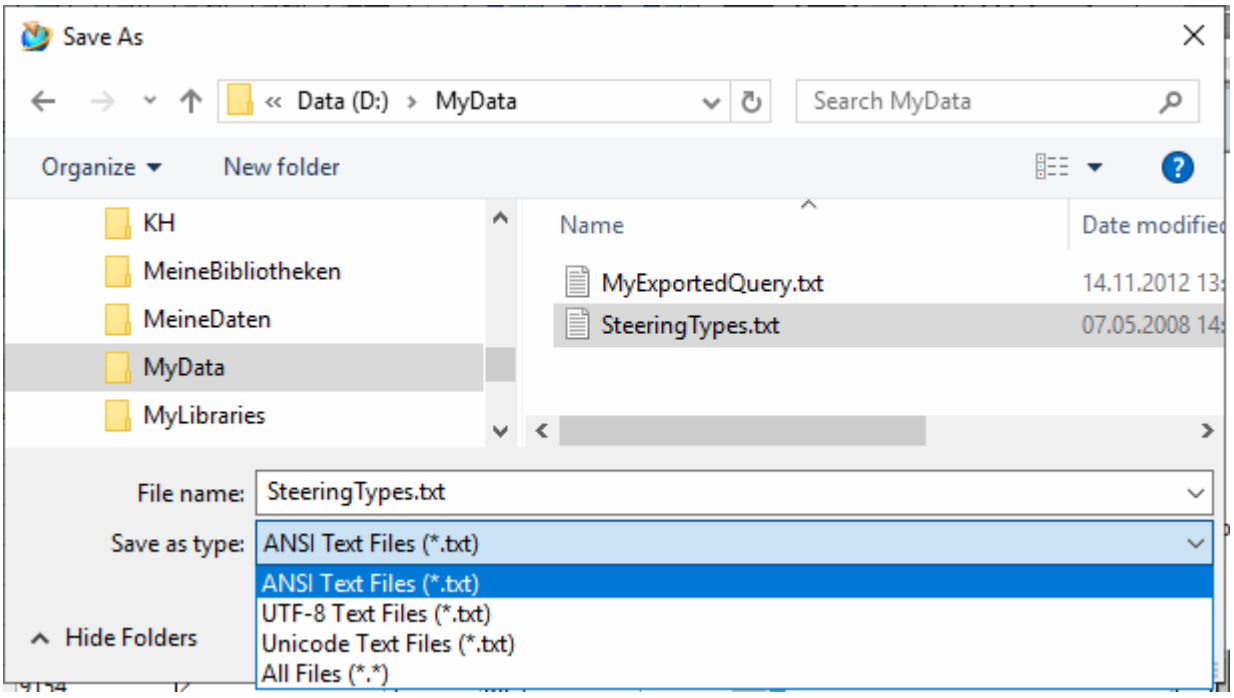
- To only save the contents of the list, without any of its formatting, click **Export > Export Text File** on the **List** ribbon tab.



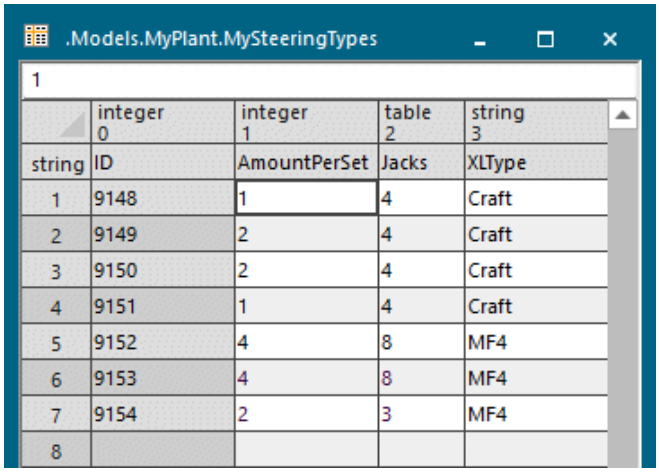
To change the settings with which *Plant Simulation* exports ASCII data, click **Export > Text File Format** on the **List** ribbon tab. Then, you can select which sign you want to use for separating columns, etc., depending on the program into which you want to import the data.

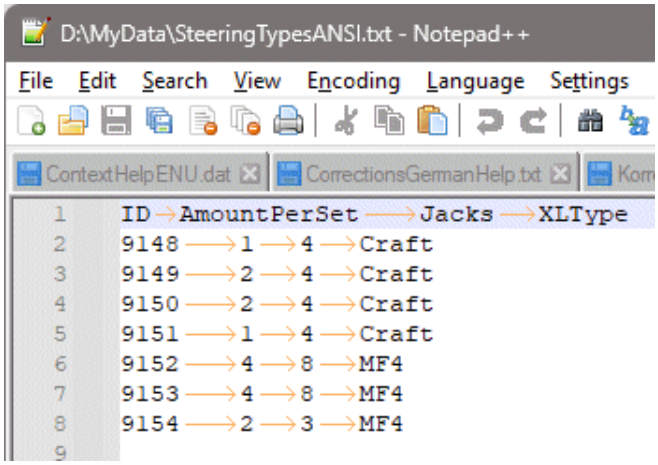


Select the encoding with which the text file will be exported in the dialog **Save As**.



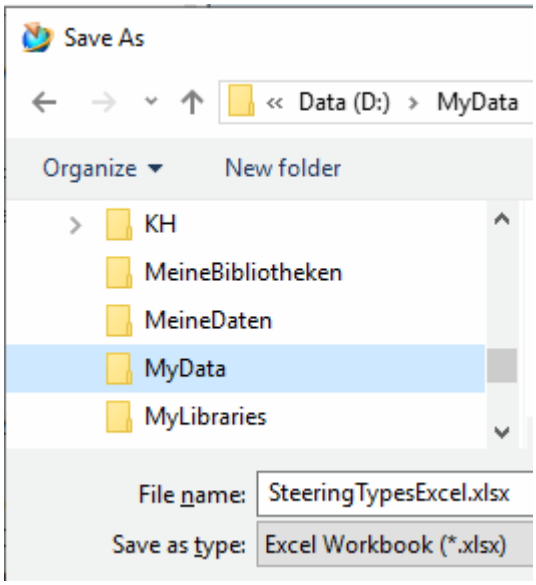
The sample table shown below, which we exported with the default settings, looks like this when we open it in a word processing program. We chose Notepad++ to show the tabs, which *Plant Simulation* exported.

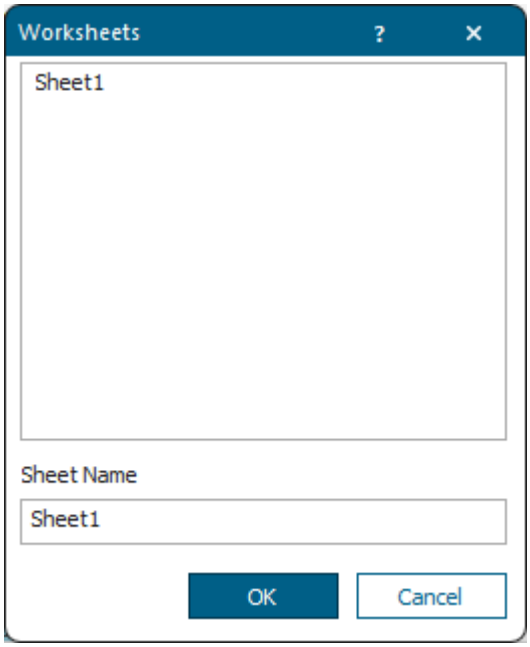




- To save the contents of the *Plant Simulation* list as an Excel worksheet, click **Export > Export Excel File** on the **List** ribbon tab.

  Enter the name of the worksheet, on which Excel opens the data, into the dialog **Worksheets**.





When *Plant Simulation* exports data to Excel, it applies these conventions:

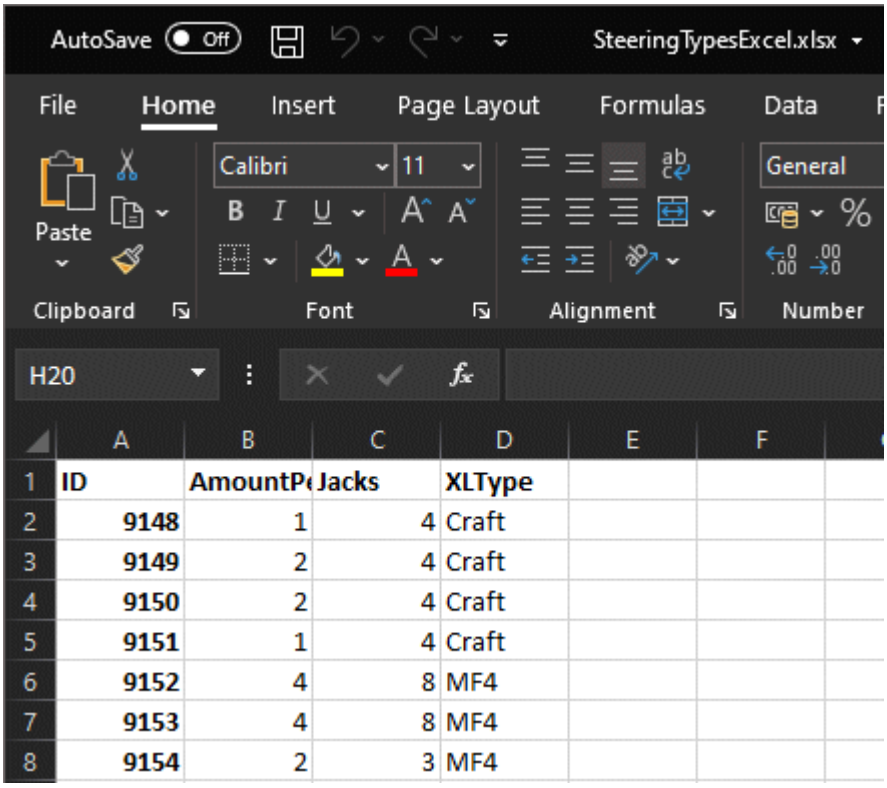
Plant Simulation data type	Excel data type	Excel format
String	String	—
Boolean	Boolean	—
Integer	Number	—
Real	Number	—

Plant Simulation data type	Excel data type	Excel format
Object	String	—
Table	String	—
List	String	—
Stack	String	—
Queue	String	—
Money	Number	—
Length	Number	—
Weight	Number	—
Speed	Number	—
Acceleration	Number	—
DateTime	Number	dd/mm/yyyy hh:mm:ss.000
Date	Number	dd/mm/yyyy
Time	Number	dd:hh:mm:ss.000

Note

Plant Simulation only exports integers in the range between -536.870.912 and 536.870.911 in the required format. *Plant Simulation* saves values outside of this range as 0.

When you open the .xlsx file in Excel, it looks like this. You might still have to adapt settings of the imported file in Excel.



	A	B	C	D	E	F
1	ID	AmountP	Jacks	XLType		
2	9148	1	4	Craft		
3	9149	2	4	Craft		
4	9150	2	4	Craft		
5	9151	1	4	Craft		
6	9152	4	8	MF4		
7	9153	4	8	MF4		
8	9154	2	3	MF4		

When *Plant Simulation* reads an Excel table, it attempts to adapt the data types of the individual columns to the available *Plant Simulation* data types. This only works, if you created the columns on the Excel worksheets, so that they only contain a single data type, meaning that, when you, for example, assign the data type **String** to a cell of a column, the entire column may only be of data type **String**.

Row 0 (zero) is an exception to this rule: When the table, which *Plant Simulation* reads, has a column index, it interprets row 0 as the column index and it will not be part of the data type designation of the columns.

Compare Export Excel File

Compare Import Data from a Microsoft Excel Worksheet

Compare _Methods for Importing and Exporting Data in Text Format

Compare _Attributes for the Text Format of Lists and Tables

Back to Work with Lists and Tables

Unshare a List or Data Table

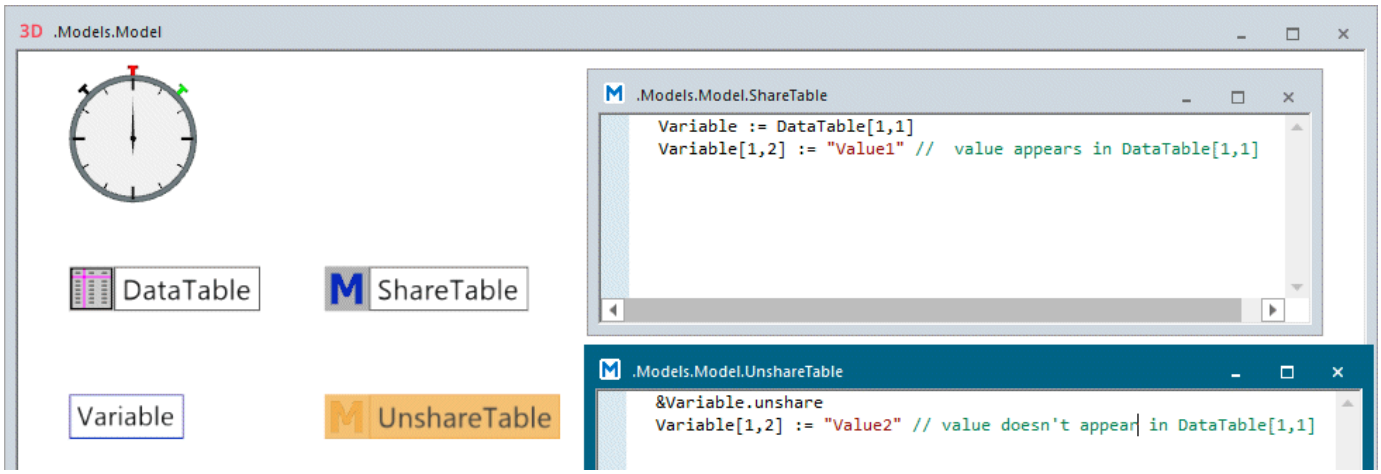
When you assign the value of a *Variable* of data types *table/list/stack/queue* to a cell in a subtable, *Plant Simulation* does not copy this value, but creates a reference to it. The *Variable* and value in the *DataTable*

then access the same value, meaning that if you change one of the values, this also changes the respective other value.

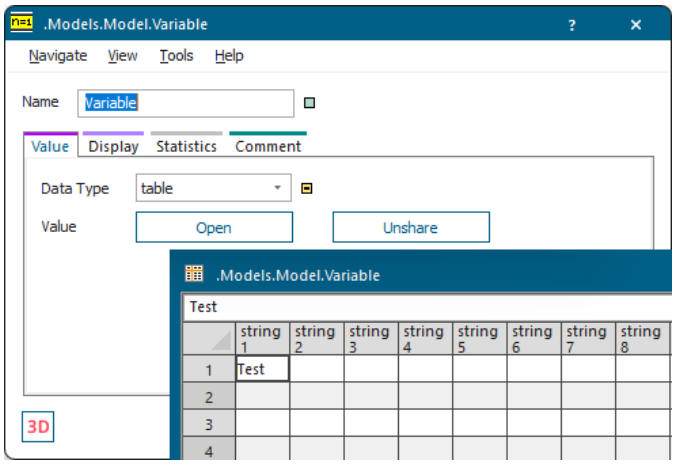
The same applies if you assign a *user-defined attribute* of data types *table/list/stack/queue* to another *user-defined attribute*. Both *user-defined attributes* then access the same list, meaning that if you change the value of one of the two attributes this also changes the value of the other attribute.

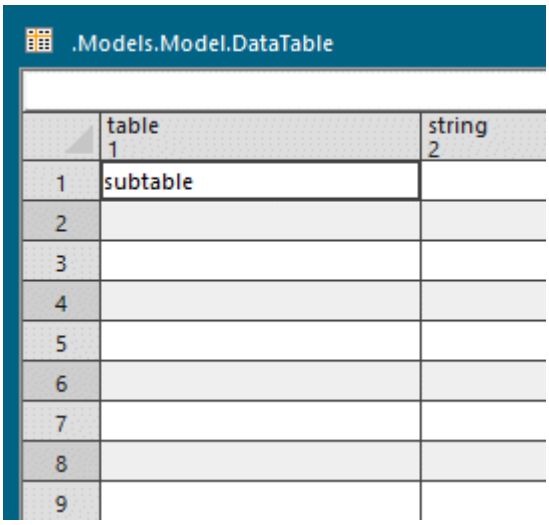
If you would like to use two different values, you can accomplish this with the method **unshare**.

Our sample model looks like this.



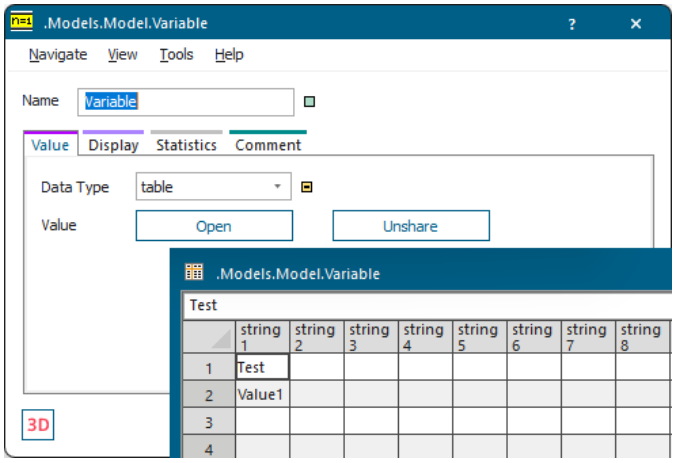
We use a *Variable* of data type *table* and a *DataTable* whose leftmost cell contains a subtable.

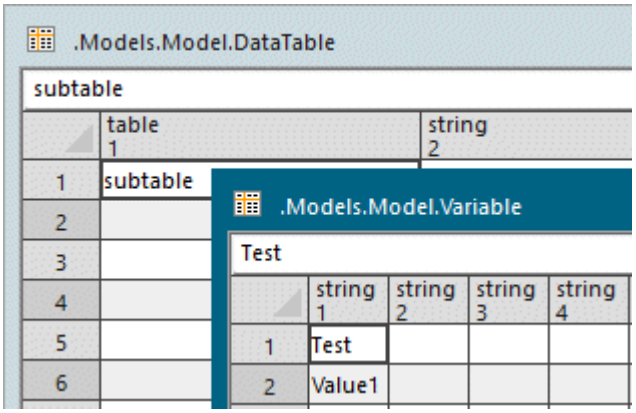




When we run the source code of our method *ShareTable* **Value1** is written to the table of the *Variable* and to the subtable of the *DataTable*. In the *DataTable* you can also unshare by clicking the button **Unshare**.

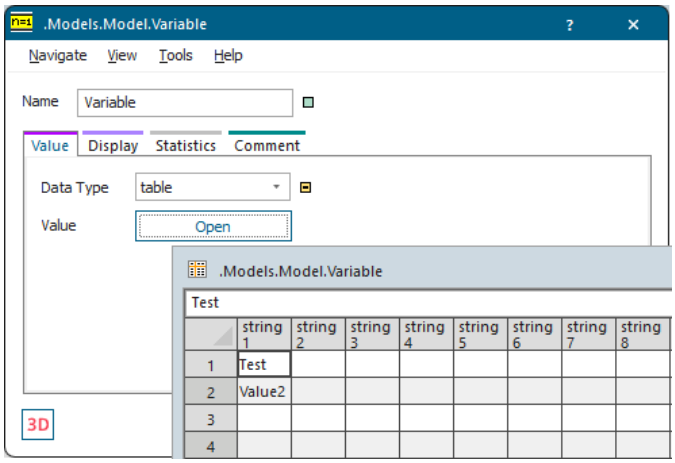
```
Variable := DataTable[1,1]
Variable[1,2] := "Value1" // Value1 appears in DataTable[1,1]
```

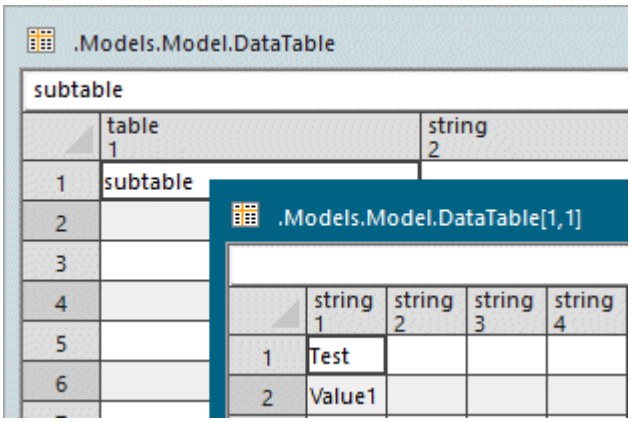




When we run the source code of our method *UnShareTable* **Value2** is written to the table of the *Variable* and **Value1** to the subtable of the *DataTable*.

```
&Variable.unshare
Variable[1,2] := "Value2" // Value2 doesn't appear in DataTable[1,1]
```

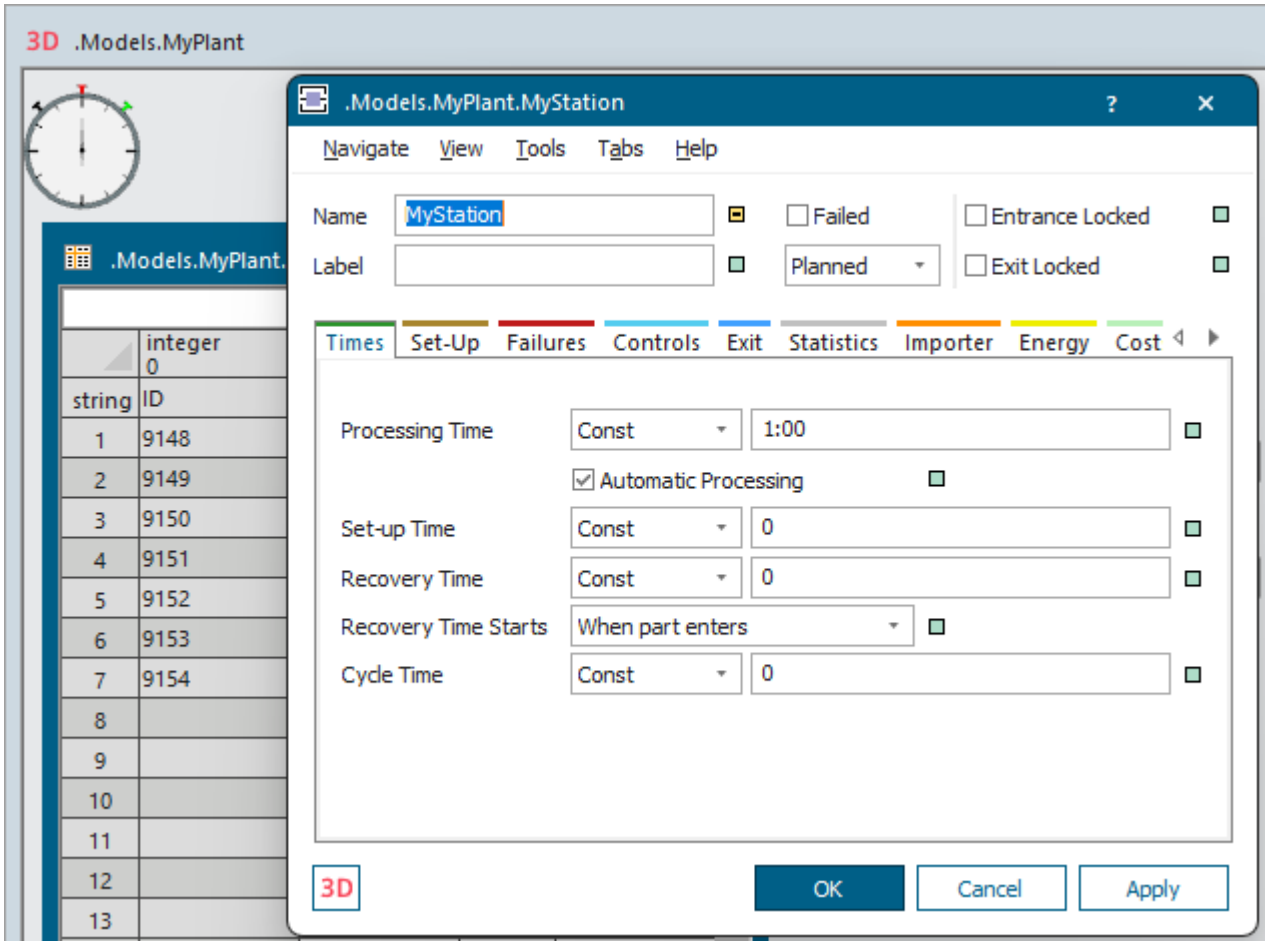




Back to Work with Lists and Tables

Open a List as a Dialog in the Foreground

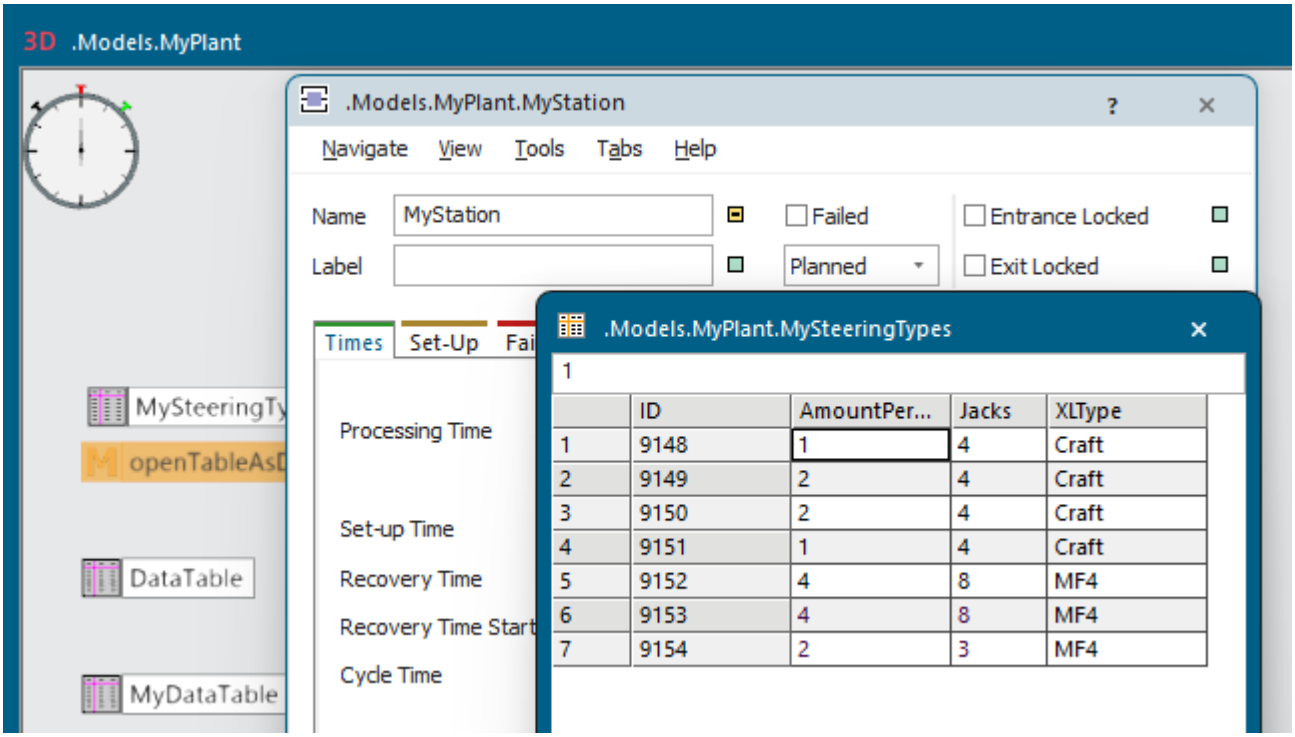
By default the windows of the objects *DataList*, *DataQueue*, *DataStack*, and *DataTable* always open in the **background** behind any open dialog boxes. This can make working with lists and tables cumbersome at times as it involves a lot of moving dialog boxes around on screen.



To prevent this, you can also open the *DataList*, the *DataQueue*, the *DataStack*, and the *DataTable* in the **foreground as a dialog box** with the method **openDialogBox**.

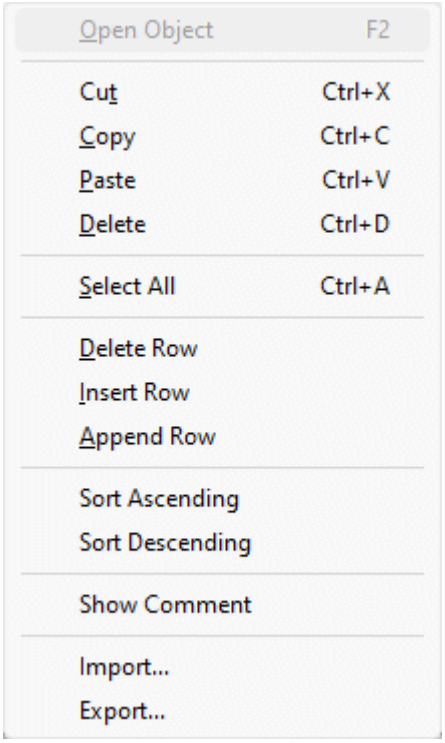
To open our table **SteeringTypes** as a dialog box in the foreground, we typed in the instructions below into our method *openTableAsDialog*.

```
SteeringTypes.openDialogBox
```



Be aware that the window of a list does not provide all of the functions which the normal list windows offers on the **List** ribbon tab.

It also offers a reduced set of functions on the context menu.



Another important difference is that the list window applies entries as you type them in, while the list opened as a window only applies them when you click **Apply** or **OK**.

Back to Tables

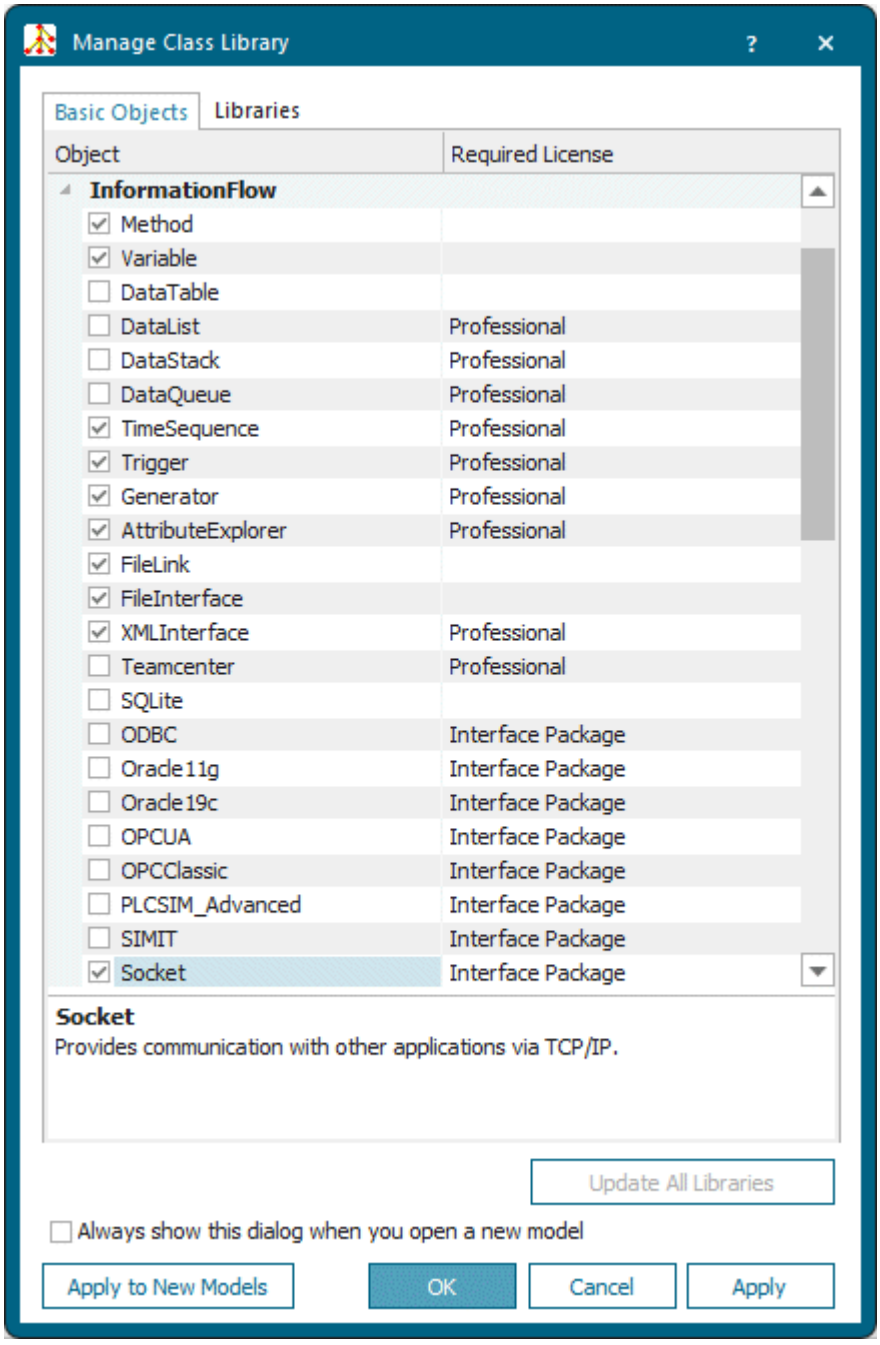
Exchange Data via a Network Socket

Socket communication is the foundation of the most widespread communication software. Sockets are point-to-point connections, established during initialization, allowing the online exchange of data. As the socket connection is directly based upon the TCP/IP protocol, it ensures fast communication without much data overhead.

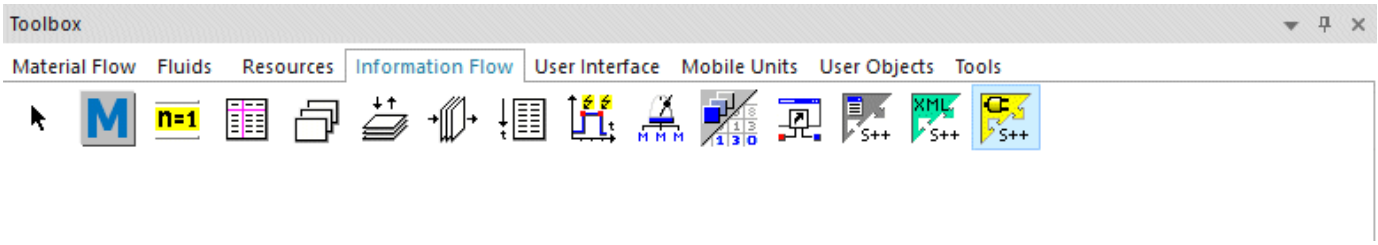
The object *Socket* provides a TCP/IP interface for *Plant Simulation*. It communicates with other applications, which also provide a socket interface.

With socket connections one process works as a server with additional processes registering as clients. *Plant Simulation* can be a client as well as a server.

To add the object **Socket** to the *Class Library*, click  **Manage Class Library** > **Basic Objects > Socket** on the **Home** ribbon tab.



You can insert the object *Socket* into your simulation model from the folder **InformationFlow** in the *Class Library* or from the toolbar **Information Flow** in the *Toolbox*.



In our sample model a *serverSocket*, located in a *Frame* of its own in *Plant Simulation*, communicates with a *client Socket*, also located in a *Frame* of its own, and vice versa.

To exchange data with the *Socket interface*, proceed as follows:

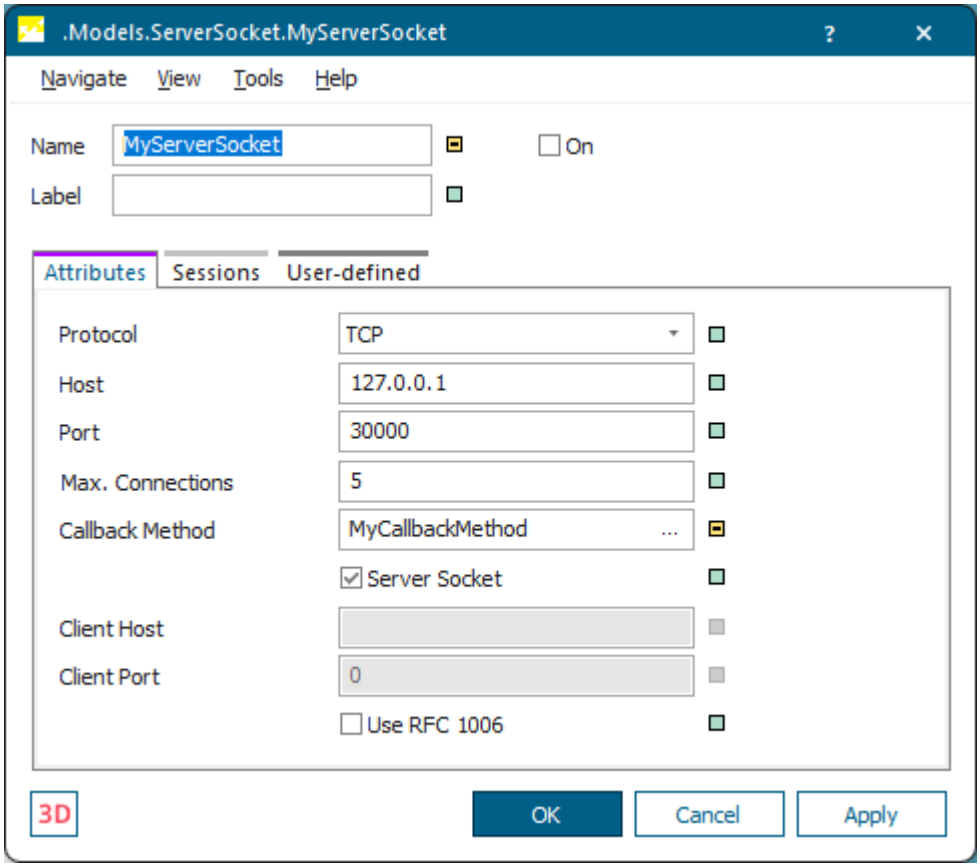
- Model the *Frame ServerSocket*.
  - Insert the objects required for the *ServerSocket* communicating with the *ClientSocket*: The *Socket* object, a **Callback Method** for the *ServerSocket*, a *Variable* each for recording the sent message and the received message, and a *Method* in which you program how to send messages.

    We named our *server Socket* object *MyServerSocket*. We used the default settings and selected our callback method, which we called *MyCallbackMethod*.

    Make sure to only select the check boxes **On** and **Server Socket** in the object *MyServerSocket*!

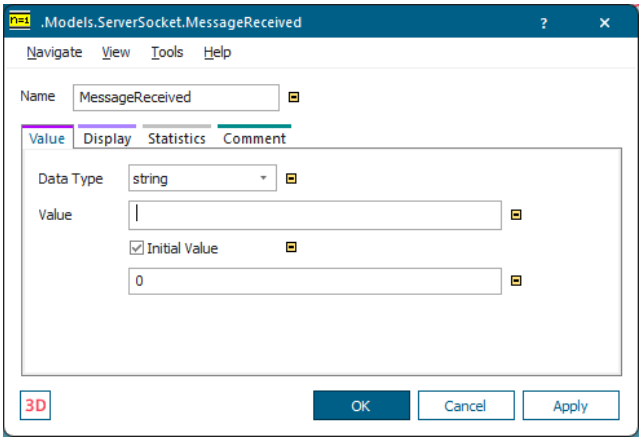
    When you select **TCP** as the **Protocol** for transmitting the data, *Plant Simulation* establishes a connection across which the data will be exchanged. The TCP protocol ensures that the data packages arrive at the destination.

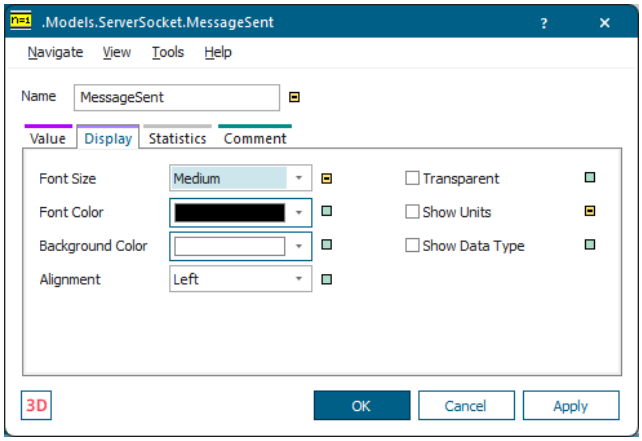
    When you select **UDP**, *Plant Simulation* can exchange data without a connection having to be established. This creates less overhead, but does not guarantee that the data actually does arrive at the destination.



- Insert the *Variables* for recording the sent message and the received message.

  We named our *Variables MessageReceived* and *MessageSent* and selected the following settings:





- Program the method with the message which you would like to send.

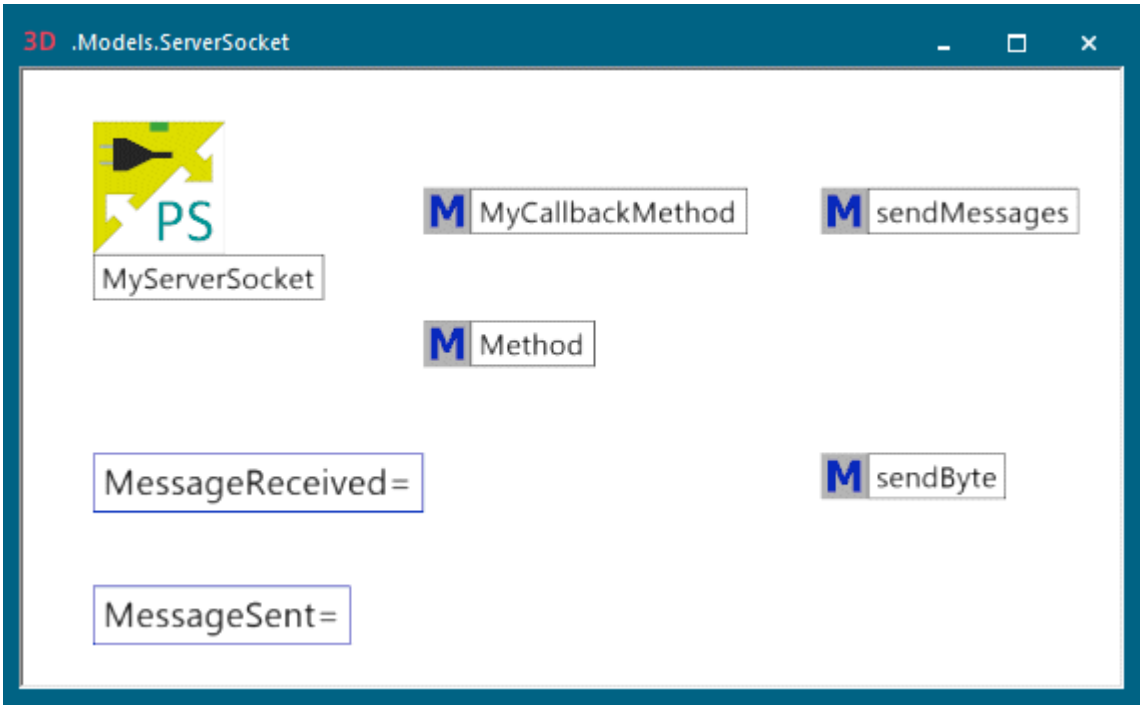
We entered the following source code to create a random number between 0 and 100, write this value to the *Variable*, which we named *MessageSent*, and send the value. We named our *method sendMessages* and entered this source code.

```
var str: string
// generates a random number between 0 and 100
str := to_str(round(z_uniform(1,0,100),1))
// writes the value of the random number to the variable 'MessageSent'
MessageSent := str
// sends the message using channel 0
MyServerSocket.write(0,str)
```

- Program the **Callback Method** which the *ServerSocket* calls when it receives data. We named our **Callback Method** *MyCallbackMethod* and entered the following source code.

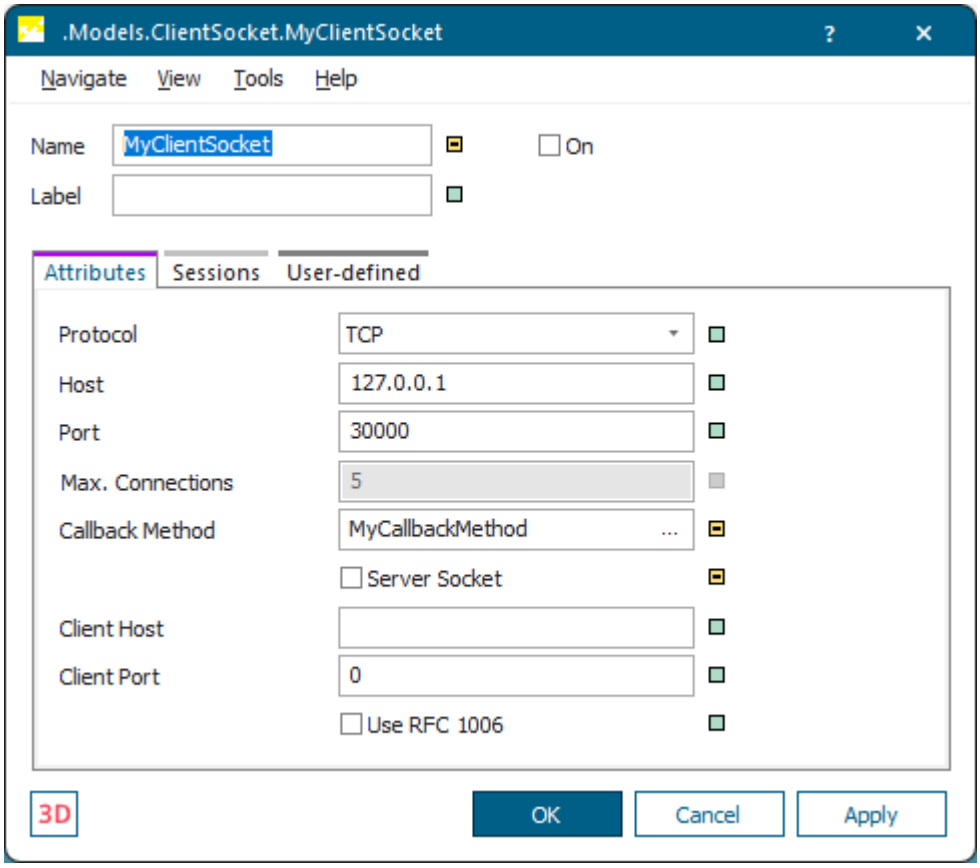
```
param SocketChannelNo: integer, SocketMessage: string
// writes the value to the global variable 'MessageReceived'
if strLen(SocketMessage) = 1
   MessageReceived := to_str(strAscii(SocketMessage)) // byte received
else
   MessageReceived := to_str(SocketMessage)           // string received
end
 // writes the message to the Plant Simulation Console
print
"-----------------------------------------------------------------"
print self
print "Message: The number ", MessageReceived, " was received at ",
sysdate
```

- The finished *Frame ServerSocket* looks like this:



- Model the *Frame ClientSocket*:
  - Insert the objects required for the *ClientSocket* communicating with the *ServerSocket*: The *Socket* object, a **Callback Method** for the *ClientSocket*, a *Variable* each for recording the sent message and the received message, and a *Method* in which you program how to send messages.
  - We named our *client Socket* object *MyClientSocket*. We used the default settings and selected our **Callback Method**, which we called *MyCallbackMethod*.

    Make sure to select the check box **On** and to clear **Server Socket** in the object *MyClientSocket*!



- Insert the *Variables* for recording the sent message and the received message.

  We named our *Variables MessageReceived* and *MessageSent* and selected the same settings as above.

  Program the method with the message which you would like to send.

  We entered the following source code to create a random number between 0 and 100, write this value to the *Variable*, which we named *MessageSent*, and send the value. We named our *method sendMessages* and entered this source code.

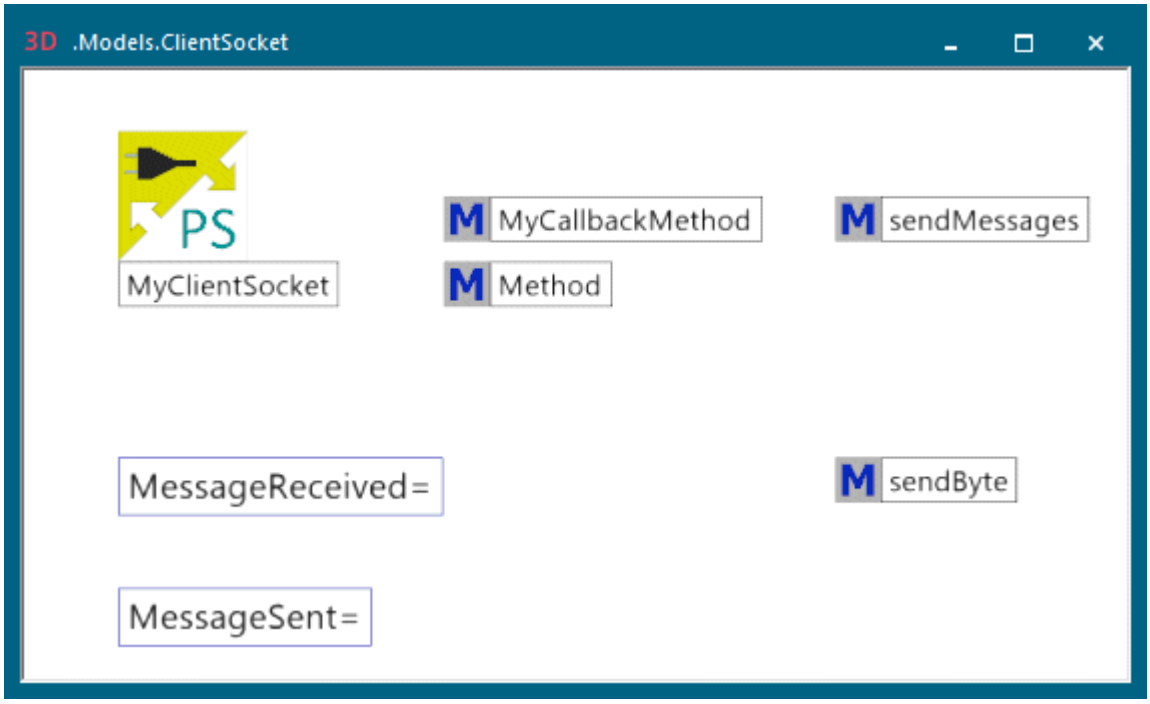
```
var str: string
// generates a random number between 0 and 100
str := to_str(round(z_uniform(1,0,100),1))
// writes the value of the random number to the variable 'MessageSent'
MessageSent := str
// sends the message using channel 0
MyClientSocket.write(0,str)
```

- Program the **Callback Method** which the *ClientSocket* calls when it receives data. We named our **Callback Method** *MyCallbackMethod* and entered the following source code.

```
param SocketChannelNo: integer, SocketMessage: string
// writes the value to the global variable 'MessageReceived'
if strLen(SocketMessage) = 1
```

```
    MessageReceived := to_str(strAscii(SocketMessage)); // byte received
else
    MessageReceived := to_str(SocketMessage); // string received
end
// writes the message to the Plant Simulation Console
print
"-------------------------------------------------------------------"
print self
print "Message: The number ", MessageReceived, " was received at ",
sysdate; )
```

- The finished *Frame ClientSocket* looks like this:



- Activate the check box **On** in *MyServerSocket* and *MyClientSocket*.
- To transmit data, click the method *sendMessages* in the *Frame ServerSocket* with the right mouse button and select **Run** on the context menu. Watch what the variables show. The variable *MessageSent* in the *Frame ServerSocket* shows that the method calculated the number **1.8** and sent it to the variable *MessageReceived* in the *Frame ClientSocket*, which naturally also shows the number **1.8**. Also check what the *Console* shows.